Protocol, part 2 of 2

Protocol, part 2 of 2

This part contains 13 notes, from `protocol/pcap-2026-10-04.md` through `protocol/v6x-clients.md`.

`protocol/pcap-2026-10-04.md`

Packet captures — 2026-10-04 sessions

Source: `server packeting 2026-10-04 *.pcapng` (12 files).

Framing

All game traffic is **2-byte big-endian length + zlib payload**. After the login frame, some servers (port 14801) switch to an encrypted stream (zlib no longer decompresses). Others (14842, 14912) stay clear zlib.

Login (cleartext, first client frame)

Session	Port	Version ID
01	14801	`GNW28015`
03	14842	`GNW22122`
04	14801	`GNW03014`
11	14912	`GNW07080`
05	14922	`GNW30123` (listserver)
06	14923	`GNW07080` (listserver guest)

Version IDs observed: `GNW28015`, `GNW22122`, `GNW03014`, `GNW07080`, `GNW30123`.

Shape: optional prefix bytes, then `GNW#####`, then length-biased account and password fields. Credentials redacted.

Short client packets (clear sessions 03 / 11)

Repeated 4–6 byte frames, newline-terminated. Examples:

```
"1-\n   "1%\n   "0f\n   "1o\n
"/[\n   "/ZA\n   "/AY\n   "0[14\n
```

One longer example includes a GANI name:

```
"*$walk/I\x96\n
```

These are the client→server updates (movement / action). Encoding is character-based (packet type + offset coords), not the provisional f32 layout. Exact field split still needs a side-by-side with the open packet-type tables.

Server packets (clear)

First bytes seen: `9`, `O`, `M`, `>`, `J`, `'`, `)`. Content includes file pushes (`sword1.png`, `head25.png`, `walk.gani`, `steps.wav`) and short updates (`Jm?z.\n` style).

Implications for Seihai

- Login can drop the provisional `PacketId::Login` blob. Use length-prefixed zlib and the `GNW#####$user)pass!!\n` shape.
- Movement is **not** raw f32. It is a short character-encoded frame. Keep `write_classic` / `write_precise` as property helpers; do not send them as the movement packet.
- Version string should be a `GNW*` id, not `"2.31"`.

Still open

- Exact movement field split (which char is x, y, dir).
- Post-login cipher on the 14801 sessions.
- Full packet-type map (the leading `"` / `M` / `>` / `J` bytes).

Update 2026-10-21

- The short `"…` client frames are `PLI_PLAYERPROPS` (id 2) prop streams: prop 15 X, 16 Y, 17 SPRITE, 10 GANI. Values are GCHAR `tiles × 2`. The decoded examples are in `login-and-walk.md`.
- The leading byte of each login is client type 0 (`0x20`). The byte after it is the gen-3 key. The 14801 sessions (2.22/2.31 tokens) switch to gen-5 partial XOR after login, which is why zlib stops inflating. 14842/14912 (2.17/1.39 tokens) stay clear because gen 2/3 server→client traffic is plain zlib. See `codec-and-framing.md`.
- First server bytes `9`, `O`, `M`, `>`, `J`, `'`, `)` are ids 25 SIGNATURE, 47 STAFFGUILDS, 45 FILEUPTODATE, 30 FILESENDFAILED, 42 NEWWORLDTIME, 7 BOARDMODIFY, and 9 PLAYERPROPS. These are from the id table only; the frames themselves have not been re-decoded. **[inf]**

`protocol/player-props.md`

Player, NPC, and baddy properties

Date: 2026-10-21

Sources: GServer-v2 `TAccount.h`, `TPlayerProps.cpp`, `TNPC.h`/`TNPC.cpp`, `TLevelBaddy.h`/`.cpp` (GPLv3, summarized). Tags: **[srv]** server source · **[2.05]** confirmed in 2.05 · **[2.22]** / **[2.31]** read from that client's prop parser · **[inf]** inferred.

A prop stream is `(GCHAR id, value)*` running to the end of the message. It appears in `PLI_PLAYERPROPS` (2), `PLO_PLAYERPROPS` (9, self), `PLO_OTHERPLPROPS` (8, `GSHORT player id` + stream), `PLO_NPCPROPS` (3, `GINT3 npc id` + stream), and `PLO_BADDYPROPS` (2, `GCHAR baddy id` + stream). There is no length per prop, so a reader must know every id's encoding up to the highest id it expects. **[srv]**

Encodings: `c` = GCHAR, `s` = GSHORT, `i3` = GINT3, `i5` = GINT5, `str` = GSTRING.

PLPROP (player)

Id	Name	Encoding	Notes
0	NICKNAME	str	
1	MAXPOWER	c	heart containers
2	CURPOWER	c	hearts × 2
3	RUPEESCOUNT	i3	gralats
4	ARROWSCOUNT	c	
5	BOMBSCOUNT	c	
6	GLOVEPOWER	c	
7	BOMBPOWER	c	
8	SWORDPOWER	c(power + 30) + str(image)	
9	SHIELDPOWER	c(power + 10) + str(image)	
10	GANI / BOWGIF	str (≥ 2.1) · pre-2.1: c(10 + len) + image, or c(bowpower) if no image	**2.05 uses the pre-2.1 form** [srv] [inf]
11	HEADGIF	c(len + 100) + image	
12	CURCHAT	str	chat bubble text
13	COLORS	5 × c	color-name indexes; order probably skin, coat, sleeves, shoes, belt (matches the GS1 `set*color` commands) [inf]
14	ID	s	
15	X	c(x × 2)	half-tile
16	Y	c(y × 2)	
17	SPRITE	c	`% 4` = direction
18	STATUS	c	bit flags, see below
19	CARRYSPRITE	c	
20	CURLEVEL	str	gmap name when on a gmap
21	HORSEGIF	str	
22	HORSEBUSHES	c	
23	EFFECTCOLORS	c	server always sends 0
24	CARRYNPC	i3	npc id
25	APCOUNTER	s(+1)	
26	MAGICPOINTS	c	
27	KILLSCOUNT	i3	
28	DEATHSCOUNT	i3	
29	ONLINESECS	i3	
30	IPADDR	i5	
31	UDPPORT	i3	
32	ALIGNMENT	c	AP 0–100
33	ADDITFLAGS	c	
34	ACCOUNTNAME	str	
35	BODYIMG	str	
36	RATING	i3	`(rating & 0xFFF) << 9 | (deviation & 0x1FF)`
37–41	GATTRIB1–5	str (≤ 223)	
42	ATTACHNPC	c(type, 0 = npc) + i3(id)	
43 / 44	GMAPLEVELX / Y	c	
45	Z	c(z + 0.5 + 50)	
46–49	GATTRIB6–9	str	
50	JOINLEAVELVL	c	1 join, 0 leave

Id	Name	Encoding	Notes
51	PCONNECTED	—	no payload; marks disconnect
52	PLANGUAGE	str	
53	PSTATUSMSG	c	index into the server's status list
54–74	GATTRIB10–30	str	
75	OSTYPE	str	2.19+ (`wind`)
76	TEXTCODEPAGE	i3	2.19+
77	UNKNOWN77	?	no client checked parses it (2.05, 2.22, 2.31); sending it aborts the rest of the stream [2.05][2.22] [2.31]
78–80	X2 / Y2 / Z2	s: `abs(v × 16) << 1`, low bit = sign	pixel precision, newer clients only
81	UNKNOWN81	c	player-list tab flag (unconfirmed). Above the highest id that 2.31 parses (80), so 2.31 and older stop reading at it [2.31]
82	COMMUNITYNAME	str	

2.05 sends props 0, 11, 13, 18, 31, and 33 together at session start, and 42, 15, 16 together on attach or move. **[2.05]** Which ids 2.05 parses depends on the stream (see "Unknown prop ids" below). **[2.05]**

STATUS bits (prop 18) [srv]

`0x01` paused · `0x02` hidden · `0x04` male · `0x08` dead · `0x10` allow weapons · `0x20` hide sword · `0x40` has spin attack.

ADDITFLAGS [srv]

`0x01` no mass message · `0x02` only show one level · `0x04` no toall · `0x08` voice disabled.

NPCPROP

Id	Name	Encoding	Id	Name	Encoding
0	IMAGE	str	18	SPRITE	c (pre-2.x clients get `% 4`)
1	SCRIPT	s(len ≤ 0x3FFF) + text	19	COLORS	5 × c
2	X	c(px / 8) = half-tiles	20	NICKNAME	str
3	Y	c(px / 8)	21	HORSEIMAGE	str
4	POWER	c	22	HEADIMAGE	c(len + 100) + image
5	RUPEES	i3	23–32	SAVE0–9	c each
6	ARROWS	c	33	ALIGNMENT	c
7	BOMBS	c	34	IMAGEPART	`s x, s y, c w, c h` (x, y clamped to 0–16000, w, h to 0–220) [2.05]
8	GLOVEPOWER	c	35	BODYIMAGE	str
9	BOMBPOWER	c	36–40	GATTRIB1–5	str
10	SWORDIMAGE	c(power + 30) + str, or c(0)	41 / 42	GMAPLEVELX / Y	c
11	SHIELDIMAGE	c(power + 10) + str, or c(0)	43	Z	c(z + 50), z −50..170 (2.22, 2.31) [2.22]
12	GANI / BOWGIF	str	44–47	GATTRIB6–9	str
13	VISFLAGS	c	48	UNKNOWN48	not parsed by 2.05, 2.22 or 2.31; aborts the stream [2.22][2.31]

Id	Name	Encoding	Id	Name	Encoding
14	BLOCKFLAGS	c	49–52	SCRIPTER, NAME, TYPE, CURLEVEL	str
15	MESSAGE	str	53–73	GATTRIB10–30	str
16	HURTDXDY	c(dx × 32 + 32), c(dy × 32 + 32)	74	CLASS	NPC-server class
17	ID	i3	75 / 76	X2 / Y2	as PLPROP X2/Y2
			77	Z2	2 bytes, decoded like X2/Y2; sets the same Z (2.31) [2.31]

The SCRIPT prop carries the GS1 source text, not bytecode, for pre-4.0 clients. 2.05 parses GS1 from text (see `npc-library/gs1-reference-205.md`). **[srv][2.05]**

BDPROP (baddy)

Id	Name	Notes
0	ID	
1 / 2	X / Y	c(tiles × 2)
3	TYPE	0 graysoldier … 9 dragon (see GS1 baddy names)
4	POWERIMAGE	c(power) + str(image); an empty image means the default for the type [2.05]
5	MODE	0 walk, 1 look, 2 hunt, 3 hurt, 4 bumped, 5 die, 6 swamp-shot, 7 hare-jump, 9 dead; the 2.22 docs add 8 spider-shoot. On mode 5 the server switches to 9 after 2 s
6	ANI	c
7	DIR	c(a × 4 + b): two directions in one byte, `a = v >> 2`, `b = v & 3` (which one is the head is not traced) [2.05]
8–10	VERSESIGHT / VERSEHURT / VERSEATTACK	str each

The baddy verses come from the `\`-separated verse line in `.graal/.zelda` level files (see `formats/level-binary.md`). **[srv]**

2.05 coverage (update 2026-10-21) [2.05]

- 2.05 knows PLPROP **0–42, 50, 51** only, and not all of them in every stream (see below).
- **Sent by the client:** 0–24 (7 and 23 are constants: bomb power * = 10, effect colors 0), 26, 31, 32, 33, 35, 37–42. **Received only:** 25, 27–30, 34, 36, 50, 51.
- 10 = bow image / weapon (pre-2.1 BOWGIF), not GANI.
- 11 head: GCHAR n<100 → `head<n>.gif`; otherwise GCHAR(len+100) + name (≤100).
- 8 sword: 0–4 = default `sword<n>.gif`; otherwise power+30 then GSTRING image.
- 9 shield: 0–3 = defaults; otherwise power+10 then GSTRING image.
- 17 SPRITE = dir + 4 × animation index (GCHAR); 0x80 in one special state.
- 15/16 X/Y = one GCHAR each, half-tiles.
- 19 CARRYSPRITE: -1 none, 0, 200, 201, or tile-based values.
- 30 IPADDR (GINT5) and 31 UDPPORT (GINT3) feed the UDP peer channel (`udp-p2p.md`).
- 36 RATING (3 bytes) = rating `b0*32 + ((b1&0x7F)>>2)`, deviation `(b1&3)*128 + b2`.
- 37–41 GATTRIB1–5 (`#P1`–`#P5`) exist in 2.05.
- 51 PCONNECTED takes no payload, but 2.05 skips one byte after it. That is harmless because the player is removed straight after, but a server should not put other props behind it.
- NPCPROP 1 (SCRIPT) is applied only while the NPC has no script yet; a second SCRIPT for the same NPC is ignored.

Unknown prop ids (2.05, checked against 2.22 and 2.31)

Every prop parser in 2.05 is a loop over `(id, value)` with a `switch` on the id. An id the switch does not know ends the loop: **the rest of the message is ignored**, and the props already applied stay applied. **[2.05]** 2.22 and 2.31 do the same. **[2.22][2.31]**

So an unknown id cannot crash these clients. The multi-byte props in the other-player stream check the remaining length first, and string props are copied with clamped lengths. The self stream does not length-check fixed-size props (3, 13, 14, 24), so a truncated one reads a few bytes of garbage, but the loop then ends. **[2.05]** The practical risk for a server is silent loss: one prop the client does not know, placed early, hides every prop after it.

Ids each 2.05 parser accepts **[2.05]**:

Stream	Accepted ids	Stops at
PLAYERPROPS (9, self)	0–42	43 and up, including 50/51. Needs at least 2 bytes left, so a lone trailing id byte is dropped
OTHERPLPROPS (8)	0, 2, 8–13, 15–21, 24, 30–42, 50, 51	**1, 3–7, 14, 22, 23, 25–29**, 43–49, 52 and up
NPCPROPS (3) and NPCWEAPONADD (33)	0–40	41 and up (GMAPLEVELX/Y, Z, …)
BADDYPROPS (2)	0–10	11 and up

The OTHERPLPROPS row matters most: a server that sends a 2.05 client another player's MAXPOWER (1), RUPEES (3), ID (14), HORSEBUSHES (22) or a counter (25–29) loses every prop after it in that message. Send those to the player themself only, or put them last.

Highest ids parsed by later clients (from their switch tables): **[2.22][2.31]**

Client	Self	Other players	NPC
2.22	74 (stops at 50, 51, 53)	76 (stops at 1, 3–7, 14, 22, 25–29)	73 (stops at 48–52)
2.31	80 (stops at 50, 51, 53, 75–77)	80 (stops at 1, 3–7, 14, 22, 25–29, 77)	77 (stops at 48–52)

No client checked knows PLPROP 77 or 81, or NPCPROP 48. NPCPROP 43 is Z (`c(z + 50)`, range −50..170) from 2.22 on, and NPCPROP 77 is Z2 in 2.31.

`protocol/PROTOCOL-CHEATSHEET.md`

Protocol Cheat Sheet — 39ster 2.1 / Seihai

Default port: **14822**

Frame

```
[u16 BE length] [zlib payload]
```

Payload = one or more messages separated by `0x0A`.

GCHAR

```
enc1(v) = (v + 32) & 0xFF
dec1(b) = (signed char)(b - 32)
```

```
encN(v, n): for i = n-1 downto 0: emit (((v >> (7*i)) & 0x7F) + 32)
```

Used for opcodes, most props, coords, IDs. Frame length and raw tiles are **not** GCHAR.

Login (first packet, unscrambled)

```
type:u8_gchar
[key:u8_gchar]          // only if type == 0 (PLAYER)
version:7|8 raw bytes
mystery:1 raw byte
namelen:u8_gchar + name
passlen:u8_gchar + pass
```

Player scramble (every later packet)

```
iterator = 0x04A80B38 initially
iterator = iterator * 0x8088405 + key
pos = (iterator & 0xFFFF) % count + 1
drop byte at pos-1
```

Every message after the login line is scrambled, including the rest of the first frame. `pos` can be 1: the junk byte then sits before the id byte. RC clients: no scramble.

Outgoing

Append message + `\n`. Compress (zlib) when ≥ 4096 bytes or on flush. Prefix 2-byte BE compressed length. Send in 1024-byte slices.

File ops (server → client)

- 100 = FILELEN
- 101 = TILES
- 102 = FILEBLOB

Start map

`level13.graal`

Note (2026-10-21)

The "player scramble" above is gs2lib **gen 3** as seen by the server, which deletes one byte per message. On the 2.05 client side the byte is inserted, and its value is `((it >> 16) & 0x7F) + 0x20`. Insertion starts with the second message. The login's "key" GCHAR comes right after the type byte. RC 1.x is also gen 3. Only the web client is unscrambled. See `codec-and-framing.md`.

`protocol/public-listserver-2026-10-25.md`

Public listserver, first observed results (2026-10-25)

Derived note. Everything here comes from the program output and one packet capture of two runs of the read-only list check, made on the user's own Linux machine on 2026-10-25 (11:50 and 11:53 JST). The account name is masked, and no game-server address or name is recorded; rows are described by type tag, field lengths and counts only. The raw capture holds login frames and is kept private. No login frame bytes, credentials, or keys appear here.

Both runs fetched the list only: no game-server login, no chat, no warp.

What the runs settle

question	answer
Which listserver ports answer	14922 and 14923 accept connections. 14921 is dead: the connection is refused at once (TCP reset to the first SYN), for any client version.
Port for classic (plain-frame) logins	14923. A classic login (one plain zlib frame holding id 0 `newmain` and the id 1 account/password message, 34 bytes on the wire) got a reply there and nothing on 14922.
Port for the gen-5 login (2.22, 2.31)	14922 and 14923 both answer.
14922 and a classic login (the 2.17 profile)	The server accepts the connection and sends nothing at all, and it does not close; the client gave up after its 5 s read timeout (twice in two runs, same result). Whether a real 2.17 client sends something else is not known.
Real 2.00-era listserver reply (overnight item 13)	Observed (14923, classic login): ONE plain zlib frame, id 0 only, 4 rows of 8 fields each, no status/site/upgrade messages, no trailing NULs. The server then waits.
Shape of the 14922 reply	Two gen-5 zlib frames (comp byte 0x04): frame 1 = id 0 list; frame 2 = id 2 status, id 3 site URL, id 5 upgrade URL. See the layout below.
Shape of the 14923 reply to a gen-5 login	Three gen-5 zlib frames: the legacy list (4 rows) first, then the same two frames as 14922. The legacy list is sent straight after the id 7 handshake frame, before the client has sent its login message.
"9 server(s) online" against 8 rows	Nothing is dropped. The id 0 count byte was 8 and 8 complete rows followed (every row 8 fields, the payload consumed exactly). The number in the status text is the server's own text and does not match the row count (it still said 9 over 12 rows on 14923).
The host with missing leading characters	The server sent it that way. The host field is 15 bytes, starts with `.`, then groups of 3, 3 and 2 digits (`.ddd.ddd.ddd.dd`). The same bytes are in the legacy list and the current list. It is not a parse error.
The name that shows `???`	The name field carries three literal `0x3F` bytes. The `?` is on the wire, not an output artifact. (A three-byte character replaced one byte for one byte is the likely cause; that part is an inference.)

Frame and field layout (derived)

Client to server, gen-5 login (2.22, 2.31, and the 2.05 profile of the user's binary): two frames. - Frame 1: plain zlib (first payload byte 0x78), 26 bytes on the wire, 18 bytes decoded: id 7, key byte, 8-byte token, list name (7 letters), LF. - Frame 2: gen-5 frame with comp byte 0x02 (no compression), 18 bytes on the wire: the id 1 message.

Client to server, classic login (2.17 and 2.00 profiles): one plain zlib frame, 34 bytes on the wire, 26 decoded: an 8-byte id 0 message (list name) and a 16-byte id 1 message (account and password, each a length-prefixed string).

Server to client, id 0 list (decoded body): `id byte`, `count byte` (GCHAR), then per row a field-count byte (always 8 here) and 8 length-prefixed strings: 1. type prefix `X` plus name (`P`, `U`, `H`, or no prefix for silver), 2. language (7 bytes, English), 3. description, 4. URL, 5. version text, 6. players (1 byte, `0`), 7. host, 8. port (5 digits). LF after the last row. Gen-5 frames have two 0x00 bytes after the final LF; the plain legacy frame does not.

Server to client, status frame (decoded, 118 bytes): id 2 status text of 60 bytes (two lines joined by CR: a welcome line and the "N server(s) online" line), id 3 site URL (20 bytes), id 5 upgrade URL (30 bytes), LF after each, two trailing 0x00 bytes.

Row counts on the wire: 14922 gen-5 list: 8 rows (count byte 8, decoded 933 bytes; wire 455). Legacy list: 4 rows (count byte 4; gen-5 wire 309 / decoded 458, or plain wire 306 / decoded 456). Status frame wire 113.

The legacy rows are the same fields in the same layout, all untagged (silver). Three of the four rows also appear in the current list, one does not; for the shared ones the host, port and version text are byte for byte equal. All other fields in legacy rows use the same language and player count values as the current list.

Timing (user's link, round trip about 0.2 s)

- 14922: the reply arrives about 0.4 s after the client's second frame, both frames at once.
- 14923, gen-5 login: the legacy list arrives within a few ms of the handshake being acknowledged (before the client sends its login frame); the two current frames arrive about 0.35 s after the client's login frame. A reader that stops waiting 0.5 s after the first list leaves very little margin.
- 14923, classic login: the single reply frame arrives about 0.2 s after the login.
- 14922, classic login: no bytes for the whole 5 s, until the client closed.

Open

- An earlier run of the 2.05 profile on 14923 failed once with "unknown gen-5 compression byte 0x78" (a plain zlib frame read by a gen-5 reader). The second run of the same command succeeded, and the capture holds only the successful one, so the frame that failed was never captured. The cause is not known.
- The binary that made the runs sent the gen-5 login for the 2.05 profile; the current main sends the classic one. So the classic login on 14923 has been observed with the 2.00 and 2.17 profiles (identical frame), and the classic 2.05 login on 14923 and 14922 follows from that, but was not run.
- A real 2.17 client's list login is unobserved; what the server wants on 14922 from 2.17 is open. A classic login to 14923 with the 2.17 profile was not tried.
- A real 2.00 client sends only id 0 and the list name (no account); Seihai's 2.00 profile sends the account too. Whether the legacy list needs the account is untested.
- Why the host has lost its leading characters, and why the status number is 9, only the server's operator can say.
- No game-server login has been made, so nothing is known yet about login frames, file requests or the tamper check from a public game server.

Applied to the version profiles

Every profile now defaults to `<list-host-reborn>` on the port that gets a list: legacy profiles (1.41 to 2.04) on 14923 and supported profiles (2.05 and later) on 14922. 14921 is dead and is not a default anywhere. The port and host the build itself carries stay in the profile as `built_in_port` and `built_in_host`. Each profile also says whether its port is `confirmed` by a public check or `assumed` from the rule: only 2.00, 2.22 and 2.31 are confirmed. The table and the reasons are in `client-internals/version-profiles/list-ports.md`.

Two cases the rule does not settle: 2.17 (a classic login on 14922 got no reply, so 14922 is the rule's port, not one a 2.17 login is known to use) and 2.05 (14922 answered the user's gen-5 2.05 binary, but a classic 2.05 login was not run).

`protocol/relay-1998.md`

The 1998 relay protocol (1st Graal to 1.1.9)

Date: 2026-10-21

This covers every client from the first Delphi beta (Sept 1998) to 1.1 / 1.1.9 (Feb 1999), plus the `Gserver.exe` bundled with 1.1.9. These builds have no login token and no compression. In the GServer-v2 numbering this is encryption generation 1. The notes come from static analysis of the shipped executables. No decompiler output is reproduced.

Summary

- TCP connections to a lobby/relay server. A captured 1.1.9 client opens two to the same port: one in the fixed game (`Graal8`) and one that follows the current level (`G8<level>`). Both pair with the other player (`client-internals/live-trace-early.md`). The 1998 binaries default to the loopback address on 14801, and the connect dialog (from 3rd Graal on) is pre-filled with the Cyberjoueur server on **14802** (a fixed IPv4 placeholder `<dialog-server-host>` in 3rd, 4th and 5th Graal, `<cyberjoueur-www>` from 6th Graal, `<list-host-classic>` in 1.1.9 and 1.1). The 4th and 5th alpha builds have no pre-filled relay host.
- Every message is a text line ending in `\n`, and the first character is the command. The read buffer is 1 KB.
- The server is a lobby in the style of Stefan Knorr's Java GServ (see `client-internals/zelda-online-1998.md`). It knows nothing about the game. It groups connections into named games, gives each one an 8-character id, and forwards `D` lines between partners.
- All game state travels inside `D` lines. The server does not stamp the sender, so each `D` payload starts with the sender's own id.
- There is no peer-to-peer mode. `TAdventureServer`, which first appears in 15th Graal, is a hostname-to-IP cache record (a name string plus an address, 12-byte instance). It matches the 15th Graal readme line about IP caching. Port 14802 is just the Cyberjoueur relay port.

Transport by build

Builds	Socket layer	Notes
1st, 2nd Graal	ICS `TWSocket` (François Piette's component), event driven	The first beta creates two client objects for the local two-player mode.
3rd Graal to 1.1.9	Plain WinSock: blocking `connect`, then `FIONBIO`, then polled `recv`	`WSAStartup(1.1)`. The host is tried with `inet_addr` first and `gethostbyname` second.
15th Graal on	Same, plus the `TAdventureServer` IP cache in front of the resolver	

Levels are not sent over this connection. Web levels are fetched over plain HTTP/1.0 (`GET <path> HTTP/1.0`) by a separate small client (`TWebClient`). The port comes from the URL: 15th Graal's built-in start level is on `<cyberjoueur-www>:7777`, the others use port 80.

Lobby lines

Client to server:

Line	Meaning
`N<game>#<name>`	Join `<game>` as a normal player. The clients always send the name `def` and send the real nickname later inside `D`.
`P<id>`	Ask to play with player `<id>`, or accept a request.
`C`	Leave the current party.
`-`	Leave the available list. The client sends it when it disconnects on purpose.
`D<data>`	Relay `<data>` to the party.

Server to client:

Line	Meaning
`I<id8>`	The id the server gives you. Ids are `0000` plus a 4-digit counter.
`+<id8>`	Player `<id8>` is in the same game. If it is the only peer the client knows, the client answers `P<id8>` straight away.
`-<id8>`	Player left. The client drops it from its peer list.
`P<id8>`	Partnership confirmed. The client then pushes its whole state to the peer (login time, position, colours, nickname, sword and shield, and so on).
`C<id8>`	The partner left the party.
`D<data>`	Relayed data. See below.
`NR` / `NL`	15th Graal on, sent by the 1.1.9 GServer right after `N`. Read statically as `NR` = the server uses Cyberjoueur levels and `NL` = local levels; with `NL` the client starts in `onlinestartlocal.graal`. A server started with local levels sent `NR` and then `NL` after every `N`. Started with "Use levels from cyberjoueur", it sent `NR` alone, and the client stayed in `onlinestart.graal` (`client-internals/live-trace-early.md`).

The 1.1.9 `Gserver.exe` also accepts `O` (join as observer), `S` (super user) and `V<id>` (observe a player), as the Java server did. It is a Delphi port of the lobby with a "Start Game Server" dialog: port 14802, and a choice between local levels and levels from Cyberjoueur. Its timer also handles a separate `Chess` game.

Game names

The game name keeps incompatible betas apart, and from 3rd Graal on it also keeps different levels apart.

Builds	Game name sent with `N`
1st, 2nd	`Zelda2` (1st Graal also has a branch that builds `Z` + level base + `2`, see `client-internals/early-clients.md`)
3rd, 4th	`Graal`
4th alpha	`Graal2` (room prefix `G2`, the same dialect as Java Graal; see `client-internals/precursors-and-fan-builds.md`)
5th, 5th alpha	`Graal3`
6th	`Graal4`
7th to 9th	`Graal5`
10th to 12th	`Graal6`
13th	`Graal7`
14th, 15th, 1.1, 1.1.9	`Graal8`

The constant name is the fallback. When the player is on a level, the client builds the name from `G<n>` plus the level's location (the URL without `http://` for a web level), so each level is effectively its own relay room. That fits the readmes' "asynchronous multiplayer system". Players only meet when they are on the same level and run a compatible build. Live runs on 2026-10-31 confirmed the two connections and the level rooms for 1st (`Zhouse12` + `Zelda2`), 3rd (`Ghouse1.graal` + `Graal`), 4th alpha (`G2level13.graal` + `Graal2`) and 5th Graal

(`G3level13.graal` + `Graal3`), and put Java Graal in one room with 4th alpha (no help) and with 5th Graal (through a proxy that renames `G2` to `G3`). See `client-internals/relay-room-bridge.md`.

`D` payloads

```
D <sid:2> <type:1> <payload>
```

`sid` is the sender id as two base-128 characters (`(c0-32)*128 + (c1-32)`). The receiver turns it back into `0000` plus 4 zero-padded digits. Numbers in the payload use the same `+32` single-character encoding as the later GCHAR. Coordinates are half tiles (`(c-32)/2`). Knock-back and fine offsets use `(c-96)/16`.

Which types each build sends:

Builds	Types
1st	`A B C E F G H L M N O P R S T W` (it can also receive `D`)
2nd to 4th	adds `Q`
4th alpha to 5th alpha	adds `I U X Y Z`
6th	adds sending `D`
7th	adds `0` to `4`
8th, 9th alpha	adds `5`
9th to 14th	adds `6`, `7`
15th, 1.1, 1.1.9	adds `8`

What each type means. Layouts come from the receiving side. Types marked *inferred* are named from the game code they call and the sounds and objects it touches, not from a capture.

Type	Payload	Meaning
`A`	x, y, dir/flags, kind, one more byte	Arrow fired (`TAdventureFlying`). In a 1.1.9 capture the steady `A` lines on `onlinestartlocal.graal` came from two level NPCs that run `shootarrowleft` and `shootarrowup` every 0.5 s. The payloads decode as x and y in half tiles at each NPC, direction 1 (left) and 0 (up). Each client runs the level's NPCs itself, so both clients sent them (`client-internals/live-trace-early.md`)
`B`	x, y, w·8+h, then w·h tiles as two base-128 chars each	Board modify
`C`	text	Chat line (the client also shows its own `showname` text)
`D`	one char per object id	Listed baddies or NPCs are dead (`compsdead`)
`E`	x, y, item index	Item laid down (`greenrupee`, `bluerupee`, `fullheart` … table)
`F`	x, y, a, b	Bomb laid (`put.wav`; the 1st Graal handler creates a `TAdventureBomb`)
`G`	x, y, item index	Item picked up (`extra.wav`)
`H`	object id, dx, dy, damage	Baddy hurt (dx and dy use the `/16` encoding)
`I`	none	The player has left the level or gone to a link. Cleared by the next `P`
`L`	level name	Current level (checked against the web-level path)
`M`	repeated 6-byte records: id, x, y, sprite, dir·4+mode, state	Baddy movement batch

Type	Payload	Meaning
`N`	text	Nickname
`O`	carried object id (`' '` = none)	Carry object
`P`	x, y, sprite	Position and animation. Also plays step sounds and the horse sound. Captured walking: x and y in half tiles, and the third byte is animation frame·4 + direction (0 up, 1 left, 2 down, 3 right), sent every 100 ms while moving
`Q`	NPC id	`playertouchsme` for an NPC on the receiver's side
`R`	x, y	Object at x,y removed (*inferred*)
`S`	count, then that many colour indices	Sprite colours (4 in 1st Graal, 5 from 4th alpha)
`T`	10 chars, base 128, little end first	Login time. The peer with the earliest time becomes the level leader and runs the baddies and NPCs
`U`	x, y, kind, id	Baddy spawned (`TComputer`)
`W`	sprite	Throws the carried object (`TAdventureCarry`)
`X`	target id	Hit or kill aimed at one player. Only the named player applies it (*inferred*)
`Y`	sword power, shield power, power	Sword and shield change (`chest.wav` path)
`Z`	none	State flag (*inferred*)
`0`	horse image, or empty	Horse on or off (`TAdventureHorse`)
`1`, `4`	x, y	Horse and bush actions (*inferred*)
`2`	dx, dy	Fireball (`fire.wav`)
`3`	len, image, script	Carrying an NPC (`TEditorPerson`). The image can be an `http://` path
`5`, `6`, `7`	none	Pose flags. The readme dates sitting and sleeping to 13th Graal (*inferred*)
`8`	registration-key string	Duplicate key check. If a peer shows the same `reg.code` key, the client says "Another player uses the same registration key like you. You will be disconnected."

The Java lobby this comes from

The lobby itself is Knorr's "GServ Standalone" (1997), shipped with source in Zelda Online 2. Its full line set (observer, super user, watch, the `+<id><name>` form, the ignored `H` score line) and the Zelda Online 2 and Java Graal room names (`Z<level>1.6`, `G2<level>`) are in `client-internals/precursors-and-fan-builds.md`.

What changed in 1.21

The relay is gone in 1.21. It is replaced by a central GServer with accounts, a version token and zlib. See `gserv-1999.md`. Several `D` types live on as numbered packets with the same meaning and similar payloads, such as board modify, bomb, item, horse and throw.

`protocol/session-opcodes-cleaned.md`

Session receive opcodes (cleaned)

Date: 2026-10-14, revised 2026-10-21

Derived from observed packet shapes, public opcode tables, and capture notes. No decompile material.

Framing

Every frame is a 16-bit big-endian length followed by that many bytes. Zlib bodies begin with `78 9c` or `78 da`. For 2.05 (gen 3), the **client** inserts one junk byte into each message after the login line, and server→client messages are plain zlib. The insertion rule is in `codec-and-framing.md`. Full-frame XOR applies only to gen 4/5 clients (2.19+).

The list client and the world client share the same zlib flush path. Only the world client adds the gen-3 insertion. **[2.05]**

Common server → client meanings

First byte	Meaning	Offline equivalent
0x20	`PLO_LEVELBOARD` (0): cached-board marker or board changes	BOARD data
0x21	`PLO_LEVELLINK` (1)	LINK
0x22	`PLO_BADDYPROPS` (2), a "compu" (baddy)	BADDY
0x23	`PLO_NPCPROPS` (3)	NPC
0x24	`PLO_LEVELCHEST` (4)	CHEST
0x25	`PLO_LEVELSIGN` (5): x, y, then sign alphabet	SIGN
0x26	`PLO_LEVELNAME` (6)	level filename
0x27	`PLO_BOARDMODIFY` (7): GCHAR x/y/w/h + GSHORT tiles	BOARD modify
0x28	`PLO_OTHERPLPROPS` (8)	—
0x2E	`PLO_PLAYERWARP` (14): x×2, y×2, level	player start
0x84	`PLO_RAWDATA` (100): GINT3 length of the next raw message	—
0x85	`PLO_BOARDPACKET` (101), behind 0x84: 8192 bytes of raw LE u16 tiles	full level load
0x86	`PLO_FILE` (102), behind 0x84: name + bytes	file or cache stream

Many other bytes exist. The full table, with 2.05 coverage, is in `packet-ids.md`, and layouts are in `packet-layouts.md`. The list above covers the records an offline session already parses from a GLEVNW01 file.

Coordinates on the wire are frequently stored as `byte - 0x20`. Spawn uses `(byte - 0x20) / 2`.

Login

The first message is GCHAR client type 0, GCHAR key (`rand & 0x7F`), the eight-character token `GNW03081`, then the account and password as GSTRINGs. The "random printable prefix" is the gen-3 key. The next message is `PLI_LANGUAGE` (37, wire 0x45), carrying the language string. Details are in `login-and-walk.md`.

The token is built from eight character immediates, not a string literal. Current gs2lib maps `GNW03081` to `CLVER_2_052`, which sorts below `CLVER_2_1`.

Offline implication

An offline session does not need these opcodes. It already loads levels from disk and applies the same records. The table is useful when online mode returns, because the same LINK, SIGN, CHEST, level name, and tile rectangle records arrive on the wire under these bytes. Wiring the offline parsers to also accept the wire form is then a thin adapter.

`protocol/udp-p2p.md`

UDP peer-to-peer channel (2.05)

Date: 2026-10-21

All facts here are from client 2.05 rev 2. **[2.05]** No open-source server implements this channel, because the server never sees it.

Setup

- At connect the client binds a UDP socket on `fixedudpport` if `fixedport` is set, otherwise on an ephemeral port. Config `noudp` turns the channel off.
- It sends its own port to the server as `PLPROP_UDPPORT` (31, GINT3). The server relays this to other players.
- A peer's address is `PLPROP_IPADDR` (30, GINT5 = 32-bit IP) plus `PLPROP_UDPPORT` (31). Both arrive through `PLO_OTHERPLPROPS`.

Datagram

`seq` (1 byte) + payload. The whole datagram is XORed with the low byte of `peerId XOR ownId`.

In code, the key is the low byte of the recipient player's id field XOR the same field on the local player object. In practice the local object's field reads **1**, so the effective key is `recipientId XOR 1`. A three-player lab capture settles this on the wire. With players 2, 3 and 4 all sending to each other, the key depended only on the destination: every datagram to player 2 used key 3, to player 3 key 2, and to player 4 key 5, whoever sent it. That is `recipientId XOR 1`, and key 5 rules out `senderId`. Every datagram decoded with `seq` counting up from 0 per sender–recipient pair. **[cap]** (An earlier two-player capture could not tell `recipientId XOR 1` from `senderId`, because with ids 2 and 3 they give the same keys.) The local player apparently never stores its own server id in that field. **[2.05][cap]**

- `seq` is per peer and wraps mod 256. A duplicate `seq` is dropped.
- There is no zlib, no length prefix, and no gen-3 insertion.
- Payload = GCHAR UDP id + fields, in the same G-type encodings as TCP.
- A datagram is accepted only from the ip:port of a known player.

Routing

A level-local action goes **UDP only** when every other player in the level has a UDP port. Otherwise it goes over UDP to the peers that have one **and** over TCP to the server.

UDP ids (own dispatcher, not PLI/PLO numbering)

Id	Meaning	Dir
0	movement: X, Y, SPRITE, STATUS, CARRYSPRITE, bow	send/recv
10 / 11	bomb add / del	send/recv
12 / 13	arrow add / fire spy	send/recv
14	explosion	send/recv
15	showimg	send/recv
16 / 17	horse add / del	send/recv
18	throw carried	send/recv
19 / 20	item add / del	send/recv
21	remote hit from GS1 `hitplayer`, sent only to the victim: knockback dx, dy and half-hearts	send/recv
22	chat (≤ 253 bytes)	send/recv
23	level name only; rewritten on receive as `PLPROP_CURLEVEL` (20). No 2.05 sender	recv
24	"entered level": X, Y, level name; sent to players in the 3×3 gmap neighbourhood	send/recv
25	hit objects	send/recv
26	trigger action	send/recv

The ids 10–20 and 25–26 line up with the matching TCP PLI meanings (`PLI_BOMBADD` 4 …), but the numbers differ. The movement fields reuse PLPROP encodings [inf: field order not documented here].

Observed on the wire [cap]

Lab, two 2.05 clients on one level, both with `noudp=false`: - **Id 0 movement:** six GCHARs: X×2, Y×2, SPRITE (direction: 0 up, 2 down and 3 right were seen), STATUS (20 in the lab), CARRYSPRITE (−1 = none), and one more GCHAR that follows the current animation. It was 55 while walking, 41 on the datagram where the player stopped, and 51 on the datagram sent at the same instant as a `sword.gani` change. As characters (+0x20) those are `W`, `I` and `S`, so the value looks like the first letter of the gani name: walk, idle, sword. **[cap][inf]** The "bow" label in the table above comes from the code reading and does not match these values. It is sent about every 100 ms while moving, then less often. - **Id 15 showimg:** four GCHARs (index, x, y, name length) followed by the image name, e.g. the AFK emoticon `emoticon_afk_stay.mng`, then a few trailing bytes. It is re-sent every ~3 s while shown. - **TCP too:** while UDP was active, both clients still sent `PLI_PLAYERPROPS` X/Y/sprite updates over TCP. When the server is in the level, the "UDP only" routing rule does not stop the TCP copy. Exactly which condition picks TCP is not traced. **[inf]**

3.04 Linux (lab, 2026-10-30) [cap]

The 3.x Torque client kept this channel byte for byte. Two 3.04 clients exchanged id 0 (movement, same seven fields and gani letter), id 22 (chat), id 26 (trigger action, e.g. a mouse click as NPC id 0, x×2, y×2, `leftmouse`, UDP only) and id 15 with a single byte 9 about every 5.1 s per peer. That last one is the periodic message 2.31 sends to the server as TCP 24. Key `recipientId XOR 1` and `seq` from 0 held for every datagram. Details: `version-profiles/lab-304-linux-2026-10-30.md`.

Payloads of ids 21, 23, 24

- **21** (hit): `GCHAR((dx+4)*16)`, `GCHAR((dy+4)*16)`, `GCHAR(halfhearts)`, then optional attacker-object bytes. `dx`/`dy` is the direction from the script's `fromx, fromy` to the victim, each component clamped to

−4..4. The receiver decodes it as `(b-0x60)/16`, and the damage as `(b-0x20)/2` capped at 1.0. The sender only fires when the victim is in the same level as the local player (it compares their level objects).
- **23**: level name (raw, no length byte). The receiver adds the prop id and length itself (`chr(20+32) + chr(len+32) + name`) and applies it as prop 20 (CURLEVEL). No sender was found in 2.05: no constant id, no literal, and no computed id feeds a send helper. A later or earlier client may send it. **[inf]**
- **24**: `GCHAR(x*2)`, `GCHAR(y*2)`, level name (raw, no length byte; the receiver adds it). It is sent when the player enters a level, to peers in the surrounding gmap levels. The receiver applies it as props 15, 16 and 20 (X, Y, CURLEVEL).

Open

- Which client sends id 23 (2.05 only receives it).
- Whether 2.1 to 2.31 kept this channel on the wire (3.04 did, see above). In 2.22, `noudp=true` is the default, and voice chat also used UDP.

`protocol/v3x-clients.md`

Graal 3.x clients: Linux 3.04 and Mac OS X 3.153

Date: 2026-10-21

Sources: the Graal 3.04 Linux installer (Makeself and a 2014 tgz repack, same game files) and the Graal 3.153 Mac OS X disk image, read statically. A third upload, a 3.151 Linux installer, is damaged (see the end). Tags: **[bin]** read from the client binaries · **[file]** read from shipped data files · **[inf]** inferred.

What these clients are

- 3.x is a **C++ rewrite on the Torque Game Engine** (GarageGames). It is not Delphi. The menus, prefs and key binds are compiled TorqueScript (`.dso`, format version 33). Levels, NPCs, GS1, ganis and the Graal network protocol are native code. **[bin][file]**
- Linux 3.04 (Jan 2004): i386 ELF, UPX-packed, stripped. Uses SDL 1.2, OpenAL and fmod. Mac 3.153 (Feb 2005): PowerPC Mach-O, Carbon, stripped. Both were built by "Linux Cyberjoueurs" (Paris). **[bin] [file]**
- Neither build has a usable symbol table. Naming comes from RTTI class names (e.g. `TGraalConnection`, `TServerListClient`, `TServerLevel`, `TServerNPC`, `TGraalScript`, `GuiGraalCtrl`), from source file names left in allocation tracking (`graal/client.cpp`, `graal/serverlist.cpp`, `graal/levels.cpp`, `graal/npcs.cpp`, `graal/descipher.cpp`, …) and from the Torque console method table. This is a different code base from the Delphi 2.x clients, so the names do not carry over to them. **[bin]**
- The 3.153 data tree adds a 3D mode (`levels3d/`, `scripts/` with Torque mission, terrain and `GameConnection` scripts). 3D servers run the Torque engine's own UDP protocol alongside the Graal TCP protocol, on the listed port + 1 (28000 is only the Torque address parser's default when no port is given; see `torque/network.md` and `torque/wire-by-build.md`). On the wire 3.153 is `Graal3D` protocol 19 with Graal's class table, not stock TGE. **[file][bin]**

Version tokens

Use	3.04 Linux	3.153 Mac	Windows counterpart
Game-server login	`G3D03014`	`G3D28015`	`GNW03014` (2.22) / `GNW28015` (2.31)

Use	3.04 Linux	3.153 Mac	Windows counterpart
Listserver	`G3D30123`	`G3D30123`	`GNW30123` (2.22, 2.31)

These two 3.x tokens carry the same date stamps as the 2.22 and 2.31 Windows tokens, with `G3D` in place of `GNW`. Earlier 3.x builds had their own dates (`G3D16053` = 3.0, `G3D27063` = 3.01 in `version-id-map.md`); the server table lists `G3D03014` as 3.041, which matches this installer's "3.04 revision 1". So 3.04 speaks the 2.22 protocol and 3.153 speaks the 2.31 protocol. As in the Delphi clients from 1.38 on, the token is assembled one character at a time, so no string literal exists. **[bin]**

Listserver

- **Host and port are not compiled in.** They come from `license/license.graal`, a short CRLF text file: the primary host `<list-host-3x>`, port `14922`, the site name, the signup and translate URLs, then `listserver2` and `listserver3` as fallbacks. **[file]**
- The file is **RSA-signed**. `license/sig.license` holds a 1024-bit RSA signature (PKCS#1 v1.5 over SHA-1, public exponent 17) as 256 hex characters. `license/sig.graal` signs the shipped (packed) executable file in the same way. The public key is embedded in the client as a hex-encoded DER key. All three signatures check out against it, so the client can tell if the host list or the executable has been edited. **[bin][file]**
- Login sequence: **[bin]**
  1. id 7: key GCHAR, then `G3D30123newmain`. This is the first message of the connection. It is sent in fixed mode 3 (zlib, no cipher, no type byte).
  2. id 1: `str account`, `str password`. In the lab run of 3.04 this came in **the same plain zlib frame** as id 7. **[cap]**
  3. The socket switches to gen 5 for what follows; the lab listserver's gen-5 reply was accepted. **[cap]**
- Server → client: 0 server list, 2 account status text (shown on the server list screen; marks the login as accepted), 4 error text (message box). 1, 3, 5 and 6 are accepted and ignored. **[bin]**
- Server list (id 0): `GCHAR count`, then per server `GCHAR fieldcount` and that many `str`. Field 0 is the name and field 5 the player count. **[bin]**
- **Category prefix on the server name.** It decides which tab lists the server: **[bin]**

Prefix	Tab
`P`	Graal Gold
`3`	Graal 3D
`H`	Hosted
`U`	not listed
none	Classic

Game-server login and cipher

- Login: id **5**, key GCHAR (`rand & 0x7F`), token, `str account`, `str password`. Then id **37** (language). Neither build sends id 39 (`PLI_TAMPERCHECK`) after login, which 2.22 and 2.31 do (3.04 confirmed on the wire, `version-profiles/lab-304-linux-2026-10-30.md`). The account and password are used as plain strings, with no `dec(b) = (~b - 10)` step as in 2.17+. **[bin]**
- **Cipher and framing are gen 5, the same as the 2.22/2.31 Windows clients.** The same LCG (`it = it * 0x08088405 + key`, seed `0x04A80B38`, one iterator per direction). Login frame in fixed mode 3, then dynamic: type 2 (raw) or 4 (zlib) under 0x30 bytes, 4 under 0x1000, otherwise 6 (bz2). Even types are partially XORed (one LCG step per 4-byte group present, at most 12 groups for type 2 and 4 for types 4 and 6; a partial last group still steps). Frame `u16be (len + 1)`, type byte, body. See `codec-and-framing.md` and `client-versions.md` (PR #8, not yet on main) for the full rule. **[bin]**

Server → client ids handled

Read from each build's main receive switch. Names from gs2lib. **[bin]**

Ids	3.04	3.153	Notes
0–25, 27–43, 45–49	✓	✓	
44 `HASNPCSERVER`	✓	·	dropped in 3.153, as in 2.31
67	·	✓	raised to script as a folder-log event
68, 69 `LARGEFILESTART` / `END`	✓	✓	
74	·	✓	raised to script as an RC-chat event
75 `PROFILE`	✓	✓	
82 `SERVERTEXT`	·	✓	raised to script as `receivetext`
84 `LARGEFILESIZE`	✓	✓	
100, 101, 102	✓	✓	
104	·	✓	
131 `NPCBYTECODE`, 132, 133	·	✓	script and class registration (`joinedclasses`, `public.` variables)
134 `GANISCRIPT`	·	✓	gani text parsed in place
150, 151, 153–156, 165, 166, 168	✓	✓	
169, 171–176, 178–181, 184, 185, 186	✓	✓	3.04 lacks 170, 177, 182 and 183, which 2.22 handles
187, 189 `MOVE2`	·	✓	

Counts: 3.04 handles 79 ids, 3.153 handles 88.

Client → server: the full sent-id sets (3.04 complete, 3.153 all but six ids), prop ranges and post-login sends are in `version-profiles/3.04-3.153.md` and `.toml`.

File formats

- **`GR-V1.04`** is `GR-V1.03` plus a heights block after the NPC section: `HEIGHTS`, then 9 lines of 9 comma-separated floats, then `HEIGHTSEND`. The client writes 1.04 only when the level has heights, otherwise 1.03. This pairs with server → client id 185 (`BOARDHEIGHTS`). The 2.22 and 2.31 Windows clients already know `GR-V1.04`, so this is not new in 3.x. **[bin]**
- **`GSERVL01`** is a second text-level magic, read by the same parser as `GLEVNW01`. Keywords: `BOARD`, `LINK`, `CHEST`, `SIGN`, `NPC`…`NPCEND`, `BADDY`…`BADDYEND`, `HEIGHTS`…`HEIGHTSEND`, and `REPLACENPCS`…`REPLACENPCSEND` (skipped by the client). **[bin]**
- **`GWEBL001`** (downloaded-level cache). The body is still DES-encrypted as in 2.05. **[bin]**
  - File: a 4-byte length, then 8-byte DES blocks, then an 8-byte checksum. The checksum is the bytewise sum (mod 256, per byte position) of the decrypted blocks. A mismatch discards the cache.
  - Each block gets a fresh 8-byte key from the client's Delphi-style generator (`seed = seed * 0x08088405 + 1`, one key byte per step). Before the read, the seed is set to `0x04A80B38 × product(level-name bytes)`. The PPC caller keeps the low byte of the new seed; 2.05 uses Delphi `Random`. Which byte is used, and the length field's byte order (read natively, so possibly big-endian on the Mac), still need checking against a real cache file. **[bin][inf]**
  - Plaintext: magic, `str` server name, two 7-bit characters of protocol version, a GINT5 modification time, `str` level name, then a `GR-V1.03`/`1.04` body. The cache is used only when the server name, version and level name match. **[bin]**

- **`GRMAP001`**: 3.x reads the same keys as the 2.22/2.31 Windows clients (`WIDTH`, `HEIGHT`, `GENERATED`, `LEVELNAMES...LEVELNAMESEND`, `GENSEED`, `LEVHEIGHT`, `LEVCHAOS`, `MINIMAPIMG`, `NOAUTOMAPPING`, `HEIGHTMAP...HEIGHTMAPEND`, `RANDOMSEEDS...RANDOMSEEDSEND`). No 3.x-only key was found. **[bin]**

The uploads, for the record

- The two Mac uploads ("3.04" and "3.15r2") are byte-identical. Both are the 3.153 image.
- The 3.04 "Obsolete" installer and the 3.04 "Fixed" tgz hold the same game files. The tgz only adds x86_64 builds of the Loki installer.
- The 3.151 Linux installer ("Broken") is 139 bytes short. Its gzip payload contains no CR LF pairs at all, where about 140 would be expected. It went through an ASCII-mode FTP transfer that turned every CR LF into LF. Only the installer metadata at the start survives: version 3.151, with the same `languages3D` / `levels3d` / `scripts*` layout as the Mac 3.153. The game binary needs a clean copy.

`protocol/v4x-clients.md`

Graal 4.x clients: Windows 4.20r8, Mac OS X 4.20r6, Windows 4.04r2

Date: 2026-10-29 (4.04r2 section added the same day, after the installer was unpacked)

Sources: five uploads, read statically. Nothing was run. Tags: **[bin]** read from the client binaries · **[file]** read from shipped data files · **[src]** read from source code shipped with an upload · **[inf]** inferred.

Upload	What it holds
Windows 4.20r8 "Packet Logger v2"	`Graal4.exe` launcher, `GraalEngine.dll` (link date January 2007), `g4_PackLog.dll`. **The engine DLL is modified by a third party** (see "Packet logger").
Windows 4.20r8 "Packet Logger v1 Only"	The Code::Blocks / MinGW project and C++ source of `g4_PackLog.dll` (June 2007). Same DLL as in v2.
Mac OS X 4.20r6	HFS+ disk image: `Graal.app` (PowerPC, September 2006, "© 2006 Linux Cyberjoueurs"), `ExeReplacer`, `license/`, `languages3D/`, `levels/`, `levels3d/gui`, `sounds/`.
Windows 4.04r2 installer	`Graal4Setup.exe`, a **Thinstall** (2003) virtualised bundle. Unpacked: `Graal4.exe`, `GraalEngine.dll` (March 2006), **`Graal3DEngine.dll`**, `fmod.dll`, `ExeReplacer_temp.exe`, `license/` and the data tree (see "Windows 4.04r2"). Stock, unmodified.
Windows 4.20r8 "Unpacked"	Password-protected 7z with encrypted headers. No password is known, so it was skipped.

So the readable game code is the **4.04r2 Windows engine** (stock), the **4.20r8 Windows engine** (third-party modified) and the **4.20r6 Mac client** (stock).

What these clients are

- On Windows, 4.x splits into two parts. **[bin]**
  - `Graal4.exe` is a **Delphi** launcher. It does the window, DirectDraw drawing, input, the web panel and the external windows (PM, player list, profile, options). It registers the `graal://` and `graal3://` URL handlers, reads `graal4config.txt` and `HKCU\Software\Graal\Client`, writes `graal4console.log`, and can start `Graal3.exe` for 3D.
  - `GraalEngine.dll` is **MSVC C++** on the same Torque-derived base classes as 3.x (`JString`, `TList`, `TGraalVar`, `GuiCanvas`, …). It holds the game, scripts, files and network, and imports WinSock and fmod. It exports about 2,000 C++ symbols, including the `graal_engineInitialize` /

`graal_setHomeDirectory` entry points and about 115 `graal_set…` calls through which the launcher hands it drawing, text, Flash-panel, window and login callbacks.

- The Mac client is one PowerPC binary built from the same engine source. It ships `ExeReplacer`, a small helper that swaps in a new executable and restarts it (it only uses `chmod` and `execvp`), for the update system below. **[bin][inf]**
- The engine's own source tree is `K:\src\graalv4\…`. Crypto is **libtomcrypt** (`crypt.c, des.c, packet.c, prime.c, rsa.c, rsa_sys.c, sha1.c`), not Crypto++ as in 3.x. The other libraries are zlib, bzip2, JasPer, giflib 4.0, libjpeg and libpng. **[bin]**
- At start-up the engine makes between 0 and 1,023 heap allocations of random size, creates the main player object, then frees them. This puts the player object at a different heap address on each run, which defeats simple memory scanners. **[bin]**

So the line is: 2.x is Delphi only, 3.x is a Torque C++ monolith, and 4.x is a Delphi front end on a C++ engine.

Login server and `license.graal`

- The host comes from `license/license.graal`, as in 3.x. The 4.20r6 file reads `<login-host-4x>`, port **14900**, `<graalonline-site>`, the signup URL, the translate URL, then `<login-host-4x-2>` and `<login-host-4x-3>`. (3.x: `<list-host-3x>:14922`.) Lines used: 0 host, 1 port, 2 site name, 3 signup URL, 4 translate URL, 5 and 6 fallbacks. **[file][bin]**
- **There is no separate listserver protocol any more.** The client connects to the license host as an ordinary game server named `Login`, with the normal game login below; the server list is served inside that session. If it fails, the next host in the list is tried. **[bin]**
- Signatures use the **same RSA-1024 key as 3.x** (PKCS#1 v1.5 over SHA-1, e = 17, the key embedded as hex DER). The Mac `sig.license` verifies against `license.graal` and `sig.graal` against the Mac executable. **[bin][file]**
- Windows adds a fourth file, **`license/sig.graallib`**, the signature of `GraalEngine.dll`. The check runs in order: exe, then DLL (Windows only), then license. Any missing file stops the client with "The license file(s) have not been found", and any bad signature stops it before the license is read. The Mac client checks exe and license the same way. **[bin]**
- The stock 4.04r2 engine confirms this: when any of the three signatures fails it shows "The license file is invalid!" and stops. All three of its shipped signatures verify against its own files.
- The packet-logger copy of the 4.20r8 engine has this check disabled by a one-byte patch. A conditional jump after the result test has become an unconditional jump, so the "bad signature" branch is never taken. This is a third-party change, not 4.20r8 behaviour. **[bin][inf]**

Version tokens

Build	Game login token	Date stamp
Windows 4.04r2	`G3D17026`	17 February 2006
Mac OS X 4.20r6	`G3D27076`	27 July 2006
Windows 4.20r8	`G3D20126`	20 December 2006

The tokens follow the 3.x pattern: `G3D`, then a `DDMMY` date. The dates fit the builds (March 2006 4.04r2 engine, September 2006 Mac build, January 2007 4.20r8 engine), as they do for `G3D16053` (3.0), `G3D03014` (3.04) and `G3D28015` (3.153). Each token is still assembled one character at a time, so there is no string literal. **[bin][inf]**

Login packet

The first packet on a connection (gen 5, fixed mode 3: zlib, no cipher, no type byte, then the socket switches to dynamic mode). **[bin]**

1. id **5**, key GCHAR, the 8-character token.
2. GCHAR-length account, GCHAR-length password. (Lengths of 224 or more are sent as 0xFF followed by the first 223 bytes.)
3. A comma-text list that identifies the machine:

Field	Windows 4.20r8	Mac 4.20r6
1	`win`	`mac`
2	MD5 hex of the Windows `DigitalProductId`	hash of the primary Ethernet MAC (IOKit)
3	MD5 hex of the MAC address (from `UuidCreateSequential`, or `UuidCreate`)	hash of 4 bytes of `stat("/etc")`
4	MD5 hex of the `C:\` volume serial	empty

Then id **37** with the language name. So 4.x sends a stable PC id at every login; 2.x and 3.x did not. **[bin]**

Saved passwords

- Accounts named `guest` or starting with `guest` are never saved. **[bin]**
- The password is stored in the registry under `password` and under `<account>_pass` as `1` followed by 48 characters. The 48 characters are a 24-byte block written as nibbles plus 0x20. The block is a length byte, up to 23 password bytes and random padding, encrypted with libtomcrypt **DES** (three 8-byte blocks). **[bin]**
- The 8-byte DES key is built on the machine: two fixed bytes, four bytes of the MAC address, and two bytes of a byte sum over `DigitalProductId`, each XORed with a constant (fixed fallback bytes when the MAC or the product id is missing). So a saved password only decrypts on the PC that saved it. Compare the 2.22 scheme in `client-internals/saved-passwords.md`. **[bin]**

Cipher and framing

Gen 5, the same as 2.22, 2.31 and 3.x. Both directions are seeded with `0x04A80B38`, the generator is `it = it * 0x08088405 + key`, and the key is a random 0–127 sent in the login. Frames are `u16be length`, a type byte, then the body: under 0x30 bytes type 2 (raw), under 0x1000 type 4 (zlib), otherwise type 6 (bzip2). Type 2 XORs at most 12 dwords and types 4 and 6 at most 4. Packets are `id + 32`, the body and `\n`. **[bin]**

Large packets: before a payload sent as id 93, the client sends id 50 with a 3-byte GINT length (capped at 0x37FFFF). Incoming id 100 sets the length of the next raw (newline-free) packet. **[bin]**

Server → client ids

Windows 4.20r8 handles 101 ids: 0–25, 27–43, 45–49, 67, 68, 69, 74, 75, 82, 84, 100, 101, 102, 104–108, 131–134, 150, 151, 153–156, 165, 166, 168–176, 178–187, 189, 190, 191, 193–196. **[bin]**

Compared with 3.153 (`v3x-clients.md`), nothing is dropped and 13 ids are new. The meanings below come from the handlers; the names follow `packet-ids.md` where it has one.

id	What the 4.20r8 handler does
105	Update package size: package name and a 5-byte GINT size.

id	What the 4.20r8 handler does
106	Update package done: fires `onUpdatePackageDownloaded`, and `onPackagesDownloadComplete` when the last one ends; if the client's own executable was in a package, the client restarts itself.
107	Board change on a **layer**: layer, x, y (each under 64), width and height (1–64), then two bytes per tile. Layer 0 also clears the level's pending changes.
108	Two subtypes. 0: the client answers with id **162**, `!` and a 2-byte value. 1: stores a value on the connection.
170	Ghost mode on or off (`GHOSTMODE`).
182	The client answers with id **44**: a comma list of the titles of all open windows (`EnumWindows`).
183	The client checks whether a given string is in that window list and answers.
190	Fires `onServerListerConnect` (the `Login` server has accepted the session).
191	A projectile (`SHOOT2`): x, y, z, three angle and speed values, a gani name with optional parameters, then a script string.
193	A 5-byte GINT time and a file name: the client checks its cached copy (mod time, then CRC).
194	Clears the client's lists of weapon and class scripts and their name index.
195	A gani name and gani text: the client loads it as a gani (server-pushed gani scripts).
196	Same layout as 193; the 4.20r8 handler reads it and does nothing else.

Mac 4.20r6 handles 99 of these: the same set without 41 and 196. Its packet switch has 97 cases (0 to 195, without 41), and, as in 4.20r8, 100 and 108 are taken before the switch; its 108 handler answers subtype 0 with id 162. (Re-read 2026-10-31; this line said 97 before, which was the switch alone.) 44 is still missing in both, as in 3.153. **[bin]**

Client → server ids

Sent from the connection code (provisional, as in 3.x: some sends pass the id in a register): **[bin]**

1, 2, 4, 5, 6, 7, 8, 9, 10, 12, 13, 14, 15, 16, 17, 18, 19, 21, 22, 23, 24, 26, 28, 29, 32, 33, 35, 36, 37, 41, 44, 50, 79, 80, 81, 92, 96, 130, 152, 154, 157, 158, 161, 162.

New compared with what 3.x showed: 44 (the window list for 182), 50 (large-packet length), 157 and 161 (file requests that carry a 5-byte GINT time), 158 (asks for a weapon by name, the `getweapon` path), 162 (the answer to 108). The others match the 2.x/3.x meanings in `packet-ids.md`.

File formats

Magic	Notes
`GR-V1.00`–`GR-V1.04`, `Z3-V1.03`, `Z3-V1.04`, `GLEVNW01`	Levels. The level loader accepts all of these magics (see `formats/level-binary.md`). No `GR-V1.05` yet: board layers exist on the wire (id 107) but not in the file format. **[bin]**
`GSERVL01`, `GWEBL001`, `GRMAP001`	As in 3.x. **[bin]**

Magic	Notes
`GRPKG001`	New: the **update package** manifest (`.gupd`). Text lines `NAME`, `FLAG`, `VERSION`, `PLATFORM` (`any`, `win`, `mac`), `ACCOUNTS` (comma list), `MODE`, `SUBPACKAGE` (another `.gupd`), `FILE`, `DESCRIPTION … DESCRIPTIONEND`, `USECHECKSUM`, `ISMAINEXECUTABLE`, `PROTECTOVERWRITE`. Files ending `.exe`, `.lexe`, `.bat` or `.com` get extra checks before they are written; folder names are checked too. Local versions are kept in `localversions.txt`. Ties in with server ids 105, 106, 193 and 196, and with `ExeReplacer` on the Mac. **[bin]**
`license.graal`, `sig.*`	See above.

Packet logger (third-party build)

[src][bin]

- `g4_PackLog.dll` exports `LogIncomingPacket(char*, unsigned)`, `LogOutgoingPacket(char*, unsigned)` and `LogGraalInfo(char*)`. It appends to `Packets_In.log`, `Packets_Out.log` and `Packets_In_and_Out.log` in the working folder. It splits the buffer on `\n` and writes each line as `[id]rest`, with id = first byte − 32. `LogGraalInfo` also writes to `Packets_In.log`, because the source defines the info log name as the same file.
- The hook is in the modified `GraalEngine.dll`. A small block added at the end of the code section loads `g4_PackLog.dll` on first use and looks up the three functions. It calls `LoadLibraryA` and `GetProcAddress` at fixed kernel32 addresses (those of Windows XP SP2), so it works on that one Windows build only. It keeps the three function pointers in spare bytes at the end of `Graal4.exe`'s data section, which only works while the exe loads at its default base.
- Only one call site is wired: the end-of-stream return in zlib's `inflate`. There it logs the engine's shared decompression buffer, with the length taken from the stream's **input** count (the compressed size). In practice:
  - Only incoming **zlib** frames are logged (type 4 and the login reply), already decrypted and decompressed. Raw (type 2) and bzip2 (type 6) frames are not logged.
  - Nothing outgoing is logged. The outgoing and info stubs are in the added block, but nothing jumps to them.
  - Each record is cut to the compressed length, so long packets come out truncated.
  - Any other zlib use (resources decompressed at start-up) fires the hook too.
- The signature-check bypass above is part of the same modification.

Windows 4.04r2

How the installer is packed

`Graal4Setup.exe` is a **Thinstall** bundle. Thinstall was Jitit's 2003 application virtualiser, later VMware ThinApp; its stub names itself `X:\thinstall\stub\ts_stub.cc`. It has two layers. **[bin]**

1. **The outer loader.** The PE header's `LoaderFlags` field holds the file offset of a 56-byte table (magic `0x82D0B188`). The table's first record gives the inner stub: offset, mapped size (324 KB) and packed size (101 KB). The loader decodes it into memory, applies relocations, resolves imports and jumps to it. The decoded image has its DOS header blanked except `e_lfanew`. The outer import table keeps only the first import of each DLL of the wrapped Delphi program, as a decoy. `TS_LOADER` in the environment makes the loader read another file.
2. **The inner stub** (Thinstall runtime, linked July 2003). It reads the rest of the table: the file count (151), and the offset, packed size and unpacked size of a compressed directory. The directory holds the runtime

settings (install path `%InstallPath%\Graal4`, log `thinstall.log`, `TS_EXECUTE_EXTERNAL`, message strings) and then one record per file: flags, name, data offset, size, two FILETIMEs, an 8-byte hash, and the packed sizes of the file's 64 KB blocks. Fourteen of the records are folders. It then hooks file APIs so the program sees the bundle as real files.

Every block, the stub and the directory included, starts with a tag byte:

Tag	Coding
`0xD9`	stored
`0xD7`	a small LZ77 bit-stream coder
`0xD8`	the same, then every byte XORed with `0xF0`
`0xDE`	an adaptive range coder (the outer stub only)

In this installer the stub is `0xDE`, the directory is `0xD7`, and the file blocks are 186 × `0xD7` and 46 × `0xD9`. The 137 files come to 7.0 MB. The extraction was checked end to end: `sig.graal` verifies against `Graal4.exe`, `sig.graallib` against `GraalEngine.dll` and `sig.license` against `license.graal`, with the 3.x key. The installer was never run; only its two decoding routines were executed, in a CPU emulator. **[bin]**

What 4.04r2 ships

- `Graal4.exe` (Delphi launcher), `GraalEngine.dll` (linked 3 March 2006, about 1,650 exports), `fmod.dll` (UPX), `ExeReplacer_temp.exe` (August 2005, the Windows twin of the Mac `ExeReplacer`), and `graalv4_installfiles.txt`, a list of the installed files. **[bin][file]**
- **`Graal3DEngine.dll`** (linked 3 March 2006): the Torque 3D world as a separate DLL (details below). It exports `loadGraal3DEngine`, `runGraal3DEngine` and `endGraal3DEngine`, and imports 623 symbols from `GraalEngine.dll`. Its source paths are `K:\src\graalv4\user\engine\…` (Torque's `console`, `core`, `dgl`, `game/net`, …). So 3D is a plug-in on top of the 2D engine in 4.x; the 4.20r8 packet-logger set does not include this DLL. **[bin]**
- `license.graal` is byte-identical to the Mac 4.20r6 one (`<login-host-4x>:14900`). `levels/`, `levels3d/gui`, `levels3d/help`, `languages3D/` and `sounds/` match the Mac 4.20r6 tree; `pics1.png` is byte-identical. **[file]**

Protocol and engine differences from 4.20r8

- Same login as 4.20r8, with token `G3D17026`: id 5, key, token, account, password, then `win` and the three MD5s (product id, MAC, `C:\` serial), then id 37. Gen 5, the same framing. **[bin]**
- `GRPKG001` update packages, `localversions.txt`, `onUpdatePackageDownloaded`, `onServerListerConnect` and the window-list answer to 182 are already there in 4.04r2. **[bin]**
- **Server → client: 98 ids**, which is 4.20r8's set without 107 (layered board change), 108 and 196. 4.04r2 handles 100 inside the main switch; 4.20r8 takes 100 and 108 before it. 4.04r2 does handle 41, so the gap in the Mac 4.20r6 set is a Mac quirk. **[bin]**
- Saved passwords: 4.04r2 keeps one `password` value; the per-account `<account>_pass` copies are new in 4.20. **[bin]**
- Engine changes visible in the strings from 4.04r2 to 4.20r8: a separate network thread (`Error creating network thread`), an XML parser (Mini-XML), `GuiBrowserCtrl`, `GuiArrayCtrl`, `GuiStretchCtrl`, reconnect and "cheat window" checks, and a better random generator (`MRandomLCG`). 4.04r2 still has a few 3D-side names in the 2D engine (`getObjectByRay`, `getdatablockvars`, `graal_setGetDatablockProperties`) and a `decryptGraalMemoryDES` routine that 4.20r8 no longer has. **[bin]**

The 3D plug-in (`Graal3DEngine.dll`)

How it is started. The 4.04r2 launcher loads `Graal3DEngine.dll` when the player picks a server whose name starts with `3` (the 3D marker in the server list). If the DLL is missing, it says "You need to download Graal3D at … for playing this server!". The launcher still registers `graal3://` against `Graal3.exe`, the 3.x name. `loadGraal3DEngine` takes the command line and reads `-connect <server name>` and `graal://` / `graal3://` links. `runGraal3DEngine` is the main loop. The engine reaches the launcher through a `graal_setStartGraal3D` callback; the launcher side of that was read from its strings only. **[bin][inf]**

It replaces the launcher as the front end. At start-up the plug-in registers its own handler for every `graal_set*` callback in `GraalEngine.dll`: drawing, text, tiles, panels, clipboard, window mode and the rest. Then it initialises the engine itself. So in 3D mode the Torque DLL owns the window and the OpenGL output. The 2D engine still runs the game logic, GS2 and the Graal connection, and draws through the plug-in (`GuiGraal3DCtrl`, 3D tiles). The 3D GUI comes from `levels3d/gui` and the `languages3D/*.cs` files, which are TorqueScript globals such as `$language::Start = "Start";`. **[bin][file]**

Two connections. For a `3` server the engine opens the **normal Graal TCP connection** on the listed port. It also asks the plug-in for a **Torque UDP connection to the same host on port + 1**. The local UDP port is `fixedudpport` from the config (default 14899) when `fixedport` is on, and any free port otherwise. The Torque address parser defaults to port 28000, the 3.x 3D port. Files still come over the Graal TCP connection (`requestFileDownload`). This comes from the code path; it has not been checked on the wire. **[bin][inf]**

Torque handshake. It is stock Torque: a connect challenge request and response, then a connect request. "Connection established", "Connection to server timed out" and "The 3D server connection has timed out" are the log lines. The connect request carries: **[bin]**

Field	Value
U32 net class group, U32 class CRC	Stock Torque. A mismatch is rejected with `CHR_INVALID`, so the server needs exactly the same list of networked classes.
string game	`Graal3D` (else `CHR_GAME`)
U32 current protocol, U32 minimum protocol	**19**. A server rejects < 19 with `CHR_PROTOCOL_LESS` and a minimum ≥ 20 with `CHR_PROTOCOL_GREATER`. The client accepts only 19 in the reply (else `CHR_PROTOCOL`). v5's plug-in uses 24.
string join password	**The account password, in clear text**. The UDP side has no Graal encryption, only Torque's own packet framing.
U32 argc, then strings	2: account name, nickname (at most 16, else `CR_INVALID_ARGS`)

The plug-in also has the server-side reader for this request. Each connection outcome fires a GS2 event on the universe: `onConnectionAccepted`, `onConnectRequestTimedOut`, `onConnectRequestRejected`, `onConnectionDropped`, `onConnectionError`. After acceptance the mission loads in Torque's three phases (datablocks, ghost objects, lighting). These are reported to GS2 as `onGraal3DLoginStep` and the datablock and ghost events. **[bin]**

Graal messages inside Torque. Two custom net events carry Graal text over the UDP link. **[bin]**

- `BigDataStartEvent`: a 16-bit message id, three strings of up to 255 characters, a 24-bit total length, and a 3-bit coding (0 raw, 1 zlib, 2 bz2).
- `BigDataChunkEvent`: the 16-bit id, an 8-bit length (at most 191), then the bytes.
- A message is four strings, `(weapon, type, option, data)`, the shape of GS2 `sendtext`. Data of 0x32 bytes or more is zlib'd, and 0x1000 or more is bz2'd. The Graal TCP framing uses nearly the same limits (0x30 and 0x1000).

- With an empty weapon the type is a command: `set` / `unset` (a player flag), `events` (lines `s <flag>` or `t <trigger>`), `addweapon` (name, GINT5 time), `weaponscript`, `deleteweapon`, `addnpc` (gives a named object its script), `setmaterials` (terrain materials), and `rpg` (→ `onRPGMessage`). With a weapon name, the weapon gets `receivetext`. Type `rc` goes to `onRCChat`, and `torque` is ignored.
- The client batches its script events per connection: `trigger` as `t …`, `getweapon` as `w …`, others as `s …`. It sends them as one `events` message.
- Other custom events: `TerrainChunkEvent`, `TerrainRequestUpdatesEvent`, `RequestReloadEvent`. The rest are stock Torque (`RemoteCommandEvent`, `SimDataBlockEvent`, `GhostAlwaysObjectEvent`, `NetStringEvent`, audio events).

Formats. Torque `.dts` shapes up to version 24 ("Shape … does not match version on server"), `.dsq` sequences, `.dif` interiors, `.ter` terrain, Quake 3 `.md3` (version 15), PNG and JPEG textures, and `.dso` compiled TorqueScript. GS2 can read datablock properties (name, type and attributes) through the engine. zlib 1.1.3. **[bin]**

Open items

- A function-level diff of the 4.04r2 and 4.20r8 engines.
- The 3D plug-in on the wire: the order of the TCP and UDP logins, and which side sends which BigData commands.
- The ids sent through a register, for a complete client → server list.
- Whether the Mac 4.20r6 build handles id 41 anywhere outside its switch (it has no case there).
- The Mac 4.20r6 send ids: a scan of `li r4, <id>` before a virtual call at vtable slot 12 (the slot the 162 answer uses) finds 0-10, 12, 13, 16-21, 25, 27-29, 32, 33, 35, 36, 38, 40, 41, 43, 45, 46, 50, 81, 93, 96, 98, 130, 152, 154-159, 161 and 162, but slot 12 is not proven to be the send call on every object, so this is a lead only.
- A clean, unmodified 4.20r8 `GraalEngine.dll` (the "Unpacked" archive, if its password turns up) to confirm the patch points by diff.

`protocol/v5v6-lab-play.md`

Graal 5.x and 6.x Windows clients on a lab server

Date: 2026-10-30. The unpacked builds from `v5v6-windows-unpacked.md` were run under Wine against a local GServer-v2 (OpenGraal GS2Emu), with a proxy in between that decodes both directions and can add server packets. This page covers what the server has to send for these clients to play, what the clients send back, and how they behave in play.

Tags:

- **[wire]** seen in the decoded traffic.
- **[shot]** seen in a screenshot.
- **[srv]** behaviour of the lab server, not of an original Graal server.
- **[inf]** inferred.

Timings come from the client's own messages. A moving client sends its position every 50 ms, so they are good to about one 50 ms tick.

The lab tools, the key scripts, the proxy logs and the screenshots are in the project folder under `graal-decompile/unpacked-installers/lab-v5v6/`.

What was run

Build	Logged in	Level drawn	Played
5.12	yes, through the proxy (see "5.12's cipher step count")	yes, after the warp	walk
5.24r3, 5.24r4	yes	yes	5.24r4: full set of tests, plus pots, stones and hiding
6.00 beta1, beta2	yes	yes	login only
5.30r3	yes (clock set back)	yes	push, lift, throw, sword
5.31r4	yes (clock set back)	after the warp below	walk
6.00r2, 6.00r4	yes (clock set back)	under Wine no: freezes after the first image. **On Windows 11 yes** (see "6.00r2 and 6.00r4 on real Windows")	on Windows: walk, hurt
6.03r7	yes	after the warp below	full set of tests, plus black stone and hiding

Lab setup, and what it took

1. **Server.** GServer-v2 runs as the game server named `Login`, on the port the clients use for the login server (14900). The world folder is the 2.22 file set: levels, `pics1.png`, heads, bodies, swords, shields and ganis.
2. **Tokens.** The stock gs2lib knows `G3D14097` (5.12), `G3D26090` (5.30–6.00r4) and `G3D0311C` (6.03r7), but not `G3D15010` (5.24 and 6.00 beta2) or `G3D03048` (6.00 beta1).
    - For an unknown token the server falls back to older file packets. The 5.24 client then saves files under names like `1.png<PNG data>…` and logs "Couldn't open … for writing". **[wire]**
    - A lab copy of the library with two token strings renamed fixes this. `G3D14097` was renamed to `G3D15010`, and `G3D3007A` to `G3D03048`. **[srv]**
    - The v5 lab renamed `G3D26090` instead (`version-profiles/lab-v5-2026-10-30.md`). Which slot does not matter for files as long as it is at or above 2.14. The rule and the byte walk of the bad name are in `file-transfer.md`.
3. **Beta expiry.** 5.30r3, 5.31r4, 6.00r2 and 6.00r4 run only with the clock set back.
    - Setting the clock back only for the start is not enough. When the clock jumps forward again, the client shows "You have been disconnected due to inactivity".
    - The clock has to stay back for the whole session. The server, on the same machine, then also runs in the past, so its respawn timers are useless in those runs. **[wire]**
4. **Gani headers.** The 2.22 ganis have no `GANI0001` first line. 88 world ganis were given one, as the 5.x bundled ganis have it. This made no visible difference by itself (see the next section).
5. **Wine.** No renderer change was needed. Mouse-click the window once so that it takes keys. Hold each key for at least 150 ms: shorter taps are missed.

5.12's cipher step count

Connected straight to GServer-v2, 5.12 logs "Received illegal data (2) from game server" and is dropped. The message means a frame decoded to nothing. **[wire]**

The cause is in 5.12's frame decoder (`Graal3DEngine.dll`, read from the decompile):

- For zlib frames (type 4) and bz2 frames (type 6), it steps the gen-5 iterator **len / 4 times, rounded down**. That is still capped at 4.
- For uncompressed frames (type 2) it rounds up, as every later build does in both cases.

So any compressed frame shorter than 16 bytes whose length is not a multiple of 4 leaves 5.12 one iterator step behind the server. From then on every frame is garbage. 5.12 encodes the same way when sending, and it sends even short bodies compressed (`v5v6-windows-unpacked.md`), so the server side goes wrong too. **[bin][wire]**

- **The proxy's fix.** For the 5.12 token, decode the client with round-down stepping and re-encode for the server with the normal rule. Towards the client, send everything zlib-compressed with round-down stepping.
- With that, 5.12 logs in, warps and plays, with no illegal-data errors. It also shows its own "Players: 1 online" window and the spaced-out window title. **[wire][shot]**
- An original 5.12-era server must have stepped the same way as 5.12. A server for a 5.12 profile has to as well.

The "Login" server mode

The first thing every build does is load "tile definitions for server Login". While the current server's name is `Login`, these clients:

- draw no characters: not the own player, not other players, not gani NPCs;
- draw no HUD;
- never request `sprites.png`, heads or bodies.

The level, chests and image NPCs are drawn, and the player walks and sends positions. A D3D9 trace of 5.24r4 shows a constant 144 draw calls per frame, with nothing added when another player is sent. So this is the client choosing not to draw characters, not a rendering fault. **[wire][shot]**

The v5 thread found the cause in the binary. The draw-layer mask is cut down while the server name starts with "login". This is the login lobby mode: the real login server is not meant to be played on.

- **Fix.** After the login burst, send **S→C 178** (server warp) `0,Lab Server,<loopback>,14900` (host field = the loopback address).
- The client drops the connection, logs in again under the new name, and then draws the player, the HUD (A/S/D boxes, hearts, rupees, bombs, arrows, the AP bar), other players and their nicknames. **[wire][shot]**
- The lab does this with the proxy. GServer-v2 only sends 178 when its listserver answers a server-info query.
- Confirmed on 5.24r4, 5.31r4 and 6.03r7. 5.12 and 5.00r7 were confirmed by the v5 thread.

Tiles in 5.31r4 and 6.03r7. Before the warp these two draw no tiles at all, only chests and image NPCs. 5.24 and 5.30r3 draw them. After the warp, with a GS2 weapon `-System` given to every player that calls `addtiledef("pics1.png", "", 0)`, both draw the full level. **The warp alone is enough:** with the `-System` weapon removed from the server and the account, 5.31r4 still drew the full level after the warp, and its log shows no script running. Before the warp it drew only chests and image NPCs, as before. **[shot]**

After the warp the clients also ask for `tiledefs<server name>.txt` (here `tiledefslab%032server.txt`, with the space written as `%032`). The lab server did not have the file. **[wire]**

The session on the wire

The same flow is seen in every build that logs in. The server side is GServer-v2's, so this shows what these clients accept, not what an original Graal server sent. **[wire]**

Server, first burst after the login frame:

- 25 (signature)
- 168
- 9 (own props)
- 194 (clear weapons)
- 34 Bomb, 34 Bow
- 43
- 33 (weapon `-gr_movement`; the GS1 script is sent empty to these clients)
- 190
- 14 (warp to `onlinestartlocal.nw`, x 30, y 30.5)
- 6 (level name)
- 100 + 101 (board)
- 39 (mod time)
- 0 (empty)
- 4 × chests
- 174
- 10
- 42 (world time, every few seconds after)
- 156 (active level)
- 3 (level NPCs)
- 179 (start message)
- 41 (server HTML message)
- 82
- 47 (staff guilds)
- 180 (status list)
- 182 (window list request)

Client:

- 37 `English` (language).
- File requests:
  - **23** (plain);
  - **34** (with the cached file's time; the server answers 45 "not modified");
  - **47** (with time, for `basepackage.gupd`).
- The server answers each with a file (100 size + 102 data), or 30 "not found", or 45.
- **44**, the window list, as the answer to 182: `Select File,Lab Server,Default IME,Default IME`.
- **2**, own props: nickname, colours, language, `win`, then the position.
- **157** with a gani name, each time the client needs a gani it has not checked yet. The server answers with 195 (the gani's attributes, e.g. `SETBACKTO`).
- **158** with a weapon name (`-System`), to fetch a GS2 weapon. The server answers 140 / 197 (bytecode and its hash).
- **24**, keep-alive, every 5.1 s.
- **38** (trigger action) with `leftmouse` when the window is clicked.

Files a played session fetches. The Themida builds bundle nothing, so they ask for all of these. The MoleBox builds bundle only GUI images, so they ask for them too.

- After the warp: `sprites.png`, `body.png`, `head0.png`, `shield1.png`, `state.png`, `sword1.png`, `wbowi1.png`, `wbomb1.png`, `pics1.png`, the ganis as they are used, and the step and action sounds.

- A bold font: `arialbd.ttf` in 5.24, `tempsitcbd.ttf` in 6.03r7.
- `basepackage.gupd`, which a minimal `GRPKG001` file with only `NAME`, `VERSION`, `PLATFORM` and `MODE` satisfies.

Play: what the client does

Tested on 5.24r4 and 6.03r7, with push, lift, throw and sword also on 5.30r3. The three builds agree on every point below. The 2.05 column is from `client-internals/player-mannerisms-live.md`.

Action	5.24r4 / 5.30r3 / 6.03r7	2.05 rev 2
walk	x2/y2 sent every 50 ms, **8 pixels (0.5 tile) per tick** straight	0.5 tile per tick
run into a bush/wall	`walk` keeps going, **push 0.85–0.90 s** after the walk started	push on the 18th blocked tick (0.9 s)
while pushing	the client sends **38 `pushed,0`** about 0.55 s after `push` starts, then every **1.0 s**	–
lift (A at a bush)	`lift` and the board change in the same tick as the key; **`carrystill` 0.30 s later**	`lift` 6 ticks, then `carrystill`
carrying	the bush is drawn over the head; the head is hidden behind it. Walking plays `carry`.	the same
throw (A while carrying)	**11** (throw) and `idle` in the same tick; carry sprite back to −1	`idle` in the same tick
sword (S)	`sword` and the bush cut (**1**, board change) in the **same tick as the swing starts**; `idle` 0.25 s later (5 ticks)	cut on the first swing frame, 5 ticks

These are all **[wire]**, plus **[shot]** for carrying.

What the bush messages look like

- **Lift and sword cut.** The client sends **1**, a board change: x 30, y 16, 2 × 2, the four tiles 677, 678, 693, 694. The server echoes it as **7** to the level. **[wire]**
- The client sends nothing for items dropped from the bush. With `bushitems = true`, the item drop is the server's. **[wire][srv]**
- **Carry sprite.** Prop 19 goes to 1 for the bush while it is carried, then back to −1 when it is thrown. **[wire]**
- **Respawn.** The server puts the bush back by sending **7** with the tiles 2, 3, 18, 19, **14.6 to 14.7 s** after the lift or cut. This is GServer's `respawntime = 15`, a server setting. These clients have no respawn timer of their own for tiles. How long an original Graal 5.x/6.x server waited is not known from this lab. **[wire][srv]**

Pots, stones and hiding

The level had a pot, a light stone and a black stone put in the player's path. The account had glove power 3 (`GLOVEP 3`). Tested on 5.24r4; the black stone and bush hiding were repeated on 6.03r7 with the same results. **[wire][shot]**

Object	Lift (C→S 1) leaves tiles	Carry sprite (prop 19)	D (hide)	Server reply
bush	677, 678, 693, 694 (stump)	1	hides	–
pot	1770, 1771, 1786, 1787 (the empty base)	5	**nothing**	S→C 22: item 5 (a heart) at the pot
light stone	1834, 1835, 1850, 1851 (a hole)	3	hides	–

Object	Lift (C→S 1) leaves tiles	Carry sprite (prop 19)	D (hide)	Server reply
black stone	1834… (the same hole)	byte 0xE9	hides	–

- **Hiding** (D while carrying a bush, light stone or black stone):
  - The pose becomes `hiddenstill`, and prop 18 (status) goes from 20 to **22**, which sets the hidden bit 0x02.
  - Only the carried object is drawn, where the player stands. Body, head and shadow are gone. **[shot]**
  - Walking hidden plays `hidden` and moves **1.5 px per tick** (y2 steps of 2 and 1 px, alternating). That is about 0.1 tile per tick, as in 2.05.
- **Leaving the hiding place.** D again, or A, goes back to `carrystill`, and status goes back to 20. S while hidden throws (C→S 11).
- **Carrying.** The pot or stone is drawn over the head, as with the bush.
- **Throwing a pot.** The heart dropped by the server stays on the pot base.
- All of this matches the 2.05 rules in `client-internals/carry-hide-and-glove.md`. The 4.04r2 lab (v4 thread) got the same carry sprites and tiles.

Grab, pull, hurt and swim

Tested on 5.24r4 and 6.03r7, and the two builds agree. For the grab test the account had glove power 1, so a black stone could not be lifted. **[wire][shot]**

Action	What the client does
grab (A against a stone it cannot lift)	`grab` in the next tick (0.04–0.05 s)
pull (Left held with A held, facing the stone)	`pull` in the next tick. The player does not move, and the stone stays put. Dust puffs are drawn at the feet.
while pulling	the client sends **38 `pulled,1`** 0.85 s after `pull` starts
hurt (an explosion, S→C 36, at the player)	`hurt` and the hearts in the same tick (3 → 2.5 for power 2). The player is pushed back **5 tiles in 5 ticks** (1 tile per tick, twice walking speed) and blinks. `idle` comes **1.54–1.55 s** after the hit.
swim (walk into deep water, tile 322)	`swim` in the same tick as the first water step. Swimming moves at walking speed (0.5 tile per tick). Only the head is drawn, with a ripple, and sideways strokes throw splashes. Stepping out goes back to `walk`.

- 5.24r4 drew the explosion as a large flame cross. In 6.03r7 the 0.15 s screenshot showed no flame. Both left a crater and scorched the stone. **[shot]**
- A hurt packet sent directly (S→C 40) changed nothing in 5.24r4, whether the attacker id was 0 or a player that had been added. Only the explosion hurt the player. **[wire]**
- `pulled,1` is the second trigger action, after `pushed,0`. **[wire]**

Chat text

Chat was set by the server (own props 9 and other-player props 8, prop 12), because these clients have no chat bar of their own. Tab, Enter and F3 open nothing, and typing in play just presses the game keys (S swings the sword). Escape opens a "Do you really want to quit Graal?" dialog. On these versions the chat bar is a server-side GUI script. **[wire][shot]**

- **Look.** There is **no bubble**. The text is drawn above the head, centred, in a light colour with a blue outline, on one line. A long line is not wrapped: it runs past the edges of the view. **[shot]**
- **Font.** 5.24r4 uses a plain sans font (its config says `font=Arial`). 6.03r7 uses a script-like font (its config says `font=Tempus Sans ITC`, `fontsize=24`). Nicknames under the feet use the same style. **[shot]**

- **Timeout.** The client cleared its own chat **2.97–2.99 s** after the server set it, sending prop 12 as empty. This happened in both builds and every time. **[wire]**

6.00r2 and 6.00r4 on real Windows

Under Wine, 6.00r2 and 6.00r4 freeze on a blue window right after they write the level cache and receive the first image. On 2026-10-30 both builds were run on a real Windows PC instead (personal machine; OS build and GPU details redacted). **[shot]**

Future runs: loopback-only labs (the loopback address / a second loopback address, in a container, VM or CI). Do not use Squirrel's own machines again.

- **Setup:**
  - a real Windows 11 PC with DirectX 9 hardware (machine identity redacted);
  - the original Themida-packed `GraalSetup.exe` from each installer archive, renamed `Graal.exe`;
  - the date faked for the game process only with NirSoft RunAsDate (2011-06-14 for r2, 2011-06-28 for r4);
  - a hosts line pointing `<login-host-4x>*` at a GServer-v2 beta4 on the same LAN (`<lan-host>`; historical — future runs use loopback).
- **No freeze.** Both builds write the level cache, receive the first image and keep running. They download every file and draw the full level, the player, baddies and the HUD. Arrow keys walk the player, and a baddy's hit takes the hearts from 3 to 2.5. **The freeze is a Wine problem**, not something in the client or the server.
- **On today's date** the packed builds write the "This is a beta release" text to `game_console.log` and exit after about 1 s, before any window opens.
- **The memory dumps don't run on Windows.** The unpacked `Graal.exe` files from `v5v6-windows-unpacked.md` crash at once on Windows (entry point 0, import table full of Wine addresses). They are for reading in Ghidra only.
- **What the server had to do:**
  - serve every file, since these builds bundle nothing: `basepackage.gupd`, `pics1.png`, the sprites, heads, bodies, swords and shields, the ganis and the sounds;
  - send the 178 warp after login. Without it the screen stays black apart from chat text. After it, the window title changes to the new server name and the client requests the images.
- **A server crash to know about.** GServer-v2 beta4 crashed (SIGSEGV) on every 6.00 login while it registered the GS1 weapon `-gr_movement` (the flaghack movement weapon). It took the GS2 bytecode branch for a weapon that has no bytecode. With that weapon removed, logins worked. **[srv]**

What the Wine runs found about the freeze, kept for reference:

- The last thing the game does is write the level cache (`weblevels/…/onlinestartlocal.nw-14900.code`) and receive the first image.
- It then stops reading the socket and never paints.
- The folder scan before login finishes ("Done.").
- Switching to OpenGL (`useopengl=1`, `-opengl`), turning wined3d CSMT off and using RGBA PNGs make no difference.
- The "This is a beta release" text is on the frozen main thread's stack, and nothing in the dumped 6.00r2/r4 code refers to it. So the expiry check runs inside Themida's own code, and it seems to run again after login. **[inf]**
- It is not the file dates. Setting every server file and the client's own files to June 2011 (with the clock set back) still freezes. **[cap]**

- Themida stops winedbg from attaching.

For Seihai profiles

- A **5.x/6.x profile** draws chat as outlined text above the head, with no bubble, and clears its own chat after about 3 s. The 5.24 font is Arial. The 6.03 font is Tempus Sans ITC.
- The movement timings above (walk 0.5 tile per tick, push at 0.9 s, lift 6 ticks, sword 5 ticks with the cut on swing start) are the same as in 2.05.
- The **only new mannerisms on the wire** are the trigger actions `pushed,0`, sent once a second while pushing, and `pulled,1`, sent 0.85 s into a pull.
- Grab and pull start in the next tick, hurt pushes the player 5 tiles in 0.25 s and lasts about 1.55 s, and swimming is as fast as walking.
- There is no built-in chat bar. A profile for these versions needs a chat input of its own, the way the original servers supplied one.
- The **server** decides the tile respawn time and bush items. A Seihai-compatible server for these clients needs to:
  - use a game server name that does not start with "login" (or warp there with 178);
  - know the tokens `G3D15010` and `G3D03048`;
  - serve `basepackage.gupd` and `tiledefs<server>.txt`;
  - provide tile definitions. On 5.31 and later the warp off the "Login" server name was enough; an `addtiledef` script is not needed.

Still open

- How a server-side chat bar script sends typed chat. That needs a GS2 GUI script like the original servers had.
- Why S→C 40 (hurt by a player) did nothing.

`protocol/v5v6-windows-unpacked.md`

Graal 5.x and 6.x Windows clients, unpacked and run

Date: 2026-10-29

`v5x-clients.md` and `v6x-clients.md` could only identify the packed Windows builds, because both wrappers have to run to unpack. This page covers those ten builds after running them under Wine, dumping them from memory, and capturing their logins against a local listener. Tags: **[bin]** read from the unpacked binaries · **[cap]** seen on the wire from the running client · **[file]** read from bundled files · **[inf]** inferred.

The unpacked files, the bundled game files, the raw captures and the scripts to redo all of it are kept in the project folder under `graal-decompile/unpacked-installers/` (see `README-v5-v6.md` there). They are not in this repo.

The builds

Build	Exe date	Wrapper	Bundled files	Engine
5.12	Oct 19 2007	MoleBox	64, incl. `Graal3DEngine.dll` and a signed `license/`	Delphi launcher + `Graal3DEngine.dll` (MSVC, `F:\graallib\…`)

Build	Exe date	Wrapper	Bundled files	Engine
6.00 beta1	Jan 8 2010	MoleBox	65	one exe (`F:\quattroengine\quattro_lib\…`)
6.00 beta2	Jan 15 2010	MoleBox	65	one exe
5.24r3	Jan 22 2010	MoleBox	65	one exe
5.24r4	Feb 1 2010	MoleBox	65	one exe
5.30r3	Oct 18 2010	Themida / WinLicense 2.x	none	one exe
5.31r4	Jan 5 2011	Themida / WinLicense 2.x	none	one exe
6.00r2	Jun 12 2011	Themida / WinLicense 2.x	none	one exe
6.00r4	Jun 26 2011	Themida / WinLicense 2.x	none	one exe
6.03r7	Nov 16 2012	Themida / WinLicense 2.x	none	one exe

Dates are the compile dates in the binaries (`__DATE__` strings and PE timestamps). **[bin]**

- The wrapper on 5.12–5.24 and on the two 6.00 betas is **MoleBox** (its runtime DLL drops as `%APPDATA%\.#\MBX@…###`). It serves the bundled files to the game through hooked file APIs, so they never touch the disk. **[bin]**
- From 5.24 on (and in the 6.00 betas) the Delphi launcher and the separate engine DLL are gone: the game is **one MSVC executable**. The source tree is renamed from `graallib` to `quattroengine\quattro_lib`. **[bin]**
- The Themida builds bundle **nothing**. They start with only the exe and fetch their base files from the server (the first requests are for `basepackage.gupd`, see below). **[cap]**
- The 6.00 betas are older than 5.24r3. 6.00 beta2 even sends the same login token as 5.24r3/r4, so the two lines shared one code base in early 2010. **[bin][cap]**

Running them

- Stock Wine 9.0 crashes the Themida stub ("An internal exception occured"). The cause is a Wine bug: `VirtualProtect` with a NULL old-protection pointer dereferences it after the call fails, where Windows just returns FALSE. With that patched, Themida runs. **[bin]**
- Named `GraalSetup.exe`, the clients open an install-folder prompt first. Named `Graal.exe` they start the game directly. **[cap]**
- **5.30r3, 5.31r4, 6.00r2 and 6.00r4 have an expiry date.** On the current date they log "This is a beta release of Graal. If it's not working on your computer then please download a new version…" and exit. With the system clock set back to their build month they run normally. The check sits in Themida-virtualised code, so it was not read. 6.03r7 has no such check. **[cap][inf]**
- The unpacked files are memory dumps for reading, not for running: on Windows they crash at once (entry point 0, Wine import addresses). To run a build on Windows, use the original packed exe. `v5v6-lab-play.md` has 6.00r2 and 6.00r4 running that way on Windows 11. **[cap]**

Login server and license

Build	Login host	License file
5.12	from `license/license.graal`: `<login-host-4x>`, port 14900, fallbacks `loginserver2`/`loginserver3`	**two files**: `license.graal` + `sig.license`. The bundled signature verifies (RSA-1024, e = 17, PKCS#1 SHA-1, the same key as 3.x–5.00).

Build	Login host	License file
5.24r3 and later	the same three hosts, compiled in	none; no `license/` path is referenced any more

5.00r7 had four license files (`license.graal` plus `sig.graal`, `sig.graallib` and `sig.license`). 5.12 only signs the license text, and from 5.24 the license mechanism is gone. All Windows builds still connect to the game server named `Login` on port 14900. **[bin][file][cap]**

Login message

Every build logs in by itself as guest once it reaches the login screen. The first frame is the gen-5 login frame: `u16be` length, zlib, no cipher, no compression byte. **[cap]**

`5` + key + token + `GCHAR len` + account + `GCHAR len` + password + a comma list:

```
%kG3D14097%guest(7Y3bdzjfwin,<md5>,<md5>,<md5>,"6.2 9200 "
```

- **The key byte is sent as a GCHAR.** The wire byte minus 32 is the cipher key (`k`, 0x6B, on the wire above means key 0x4B). Wire bytes above 0x7F occur (0x98, 0x99), since the key itself goes up to 0x7F. **[cap]**
- **Guest login:** account `guest`, password **`7Y3bdzjf`** in every Windows build here. This is the plain form of the 8-byte guest password the Linux 6.x builds decrypt at run time. **[cap][inf]**
- **Machine fields:** `win`, then the MD5 hex of the Windows product id, of the network adapter MAC and of the `C:\` volume serial, as in 5.00r7, then **a fifth field**, the quoted Windows version `"<major>.<minor> <build> <service pack>"` (here `"6.2 9200 "`, which is what Windows 8 and later report to an exe without a manifest). The product-id hash differs between 5.12 and 5.24+, because 5.24 also reads `DigitalProductId4`. **[cap][bin]**
- Then **37** (language, `English`), and file requests: **23** `basepackage.gupd` (twice), then `arial.ttf`, `bigmap.txt` or `minimap.txt` depending on the build, and **34** (file request with a modification time) for `guigames_arcmessage2.png`. **[cap]**

Tokens

Build	Token	Platform field
5.00r7 (from `v5x-clients.md`)	`G3D22067`	`win`
5.12	**`G3D14097`**	`win`
6.00 beta1	**`G3D03048`**	`win`
6.00 beta2, 5.24r3, 5.24r4	**`G3D15010`**	`win`
5.30r3, 5.31r4, 6.00r2, 6.00r4	**`G3D26090`**	`win`
6.03r7 Windows	**`G3D0311C`**	`win`
6.01r1 Linux (from `v6x-clients.md`)	`G3D0108A`	`linux`
6.03r7 Linux (from `v6x-clients.md`)	`G3D0511C`	`linux`

- The token belongs to the protocol revision, not the release: four builds over nine months (5.30r3 to 6.00r4) share `G3D26090`. **[cap]**
- The digits read as a day and month shortly before each build (`G3D26090` = 26 Sep for an Oct 2010 build, `G3D15010` = 15 Jan for Jan 2010 builds), matching the 2.x `GNW` tokens. The fifth character is a year digit up to 2010 (7, 8, 0) and a letter in 6.01/6.03. **[inf]**
- **Windows 6.03r7 and Linux 6.03r7 send different tokens** (`G3D0311C` and `G3D0511C`). A server that accepts one does not necessarily accept the other. **[cap][bin]**

Cipher and framing

Gen 5, unchanged, confirmed by decoding the captures: seed `0x04A80B38`, `it = it*0x08088405 + key`, compression byte after the length, partial XOR of up to 12 dwords (`0x02`) or 4 dwords (`0x04`/`0x06`), one iterator step per group present. Every later frame of all ten captures decodes and decompresses. **[cap]**

5.12 sends even short bodies zlib-compressed (`0x04`); 5.24r3 and later send them uncompressed (`0x02`), like 2.22. **[cap]**

No build here has the 6.11 connector, `setprotocol` or the script-set id tables. LibTomCrypt (RC4, AES and others, "Ciphers built-in") is linked into all of them, including 5.12, but nothing uses it for the game socket. **[bin]**

Server → client ids

From each build's receive dispatcher (the switch on the packet id that also reaches the `onServerListerConnect` handler, S→C 190).

Builds	Ids	Set
5.12	100	0–25, 27–43, 45–49, 67–69, 74, 75, 82, 84, 101, 102, 104–107, 131–134, 140, 150, 151, 153–156, 165, 166, 168–176, 178–187, 189–191, 193–195, 197 (identical to 5.00r7)
6.00 beta1, 6.00 beta2, 5.24r3, 5.24r4	97	0–25, 27–43, 45–49, 67–69, 74, 82, 84, 101, 102, 104–107, 109, 131, 134, 140, 150, 151, 153–156, 165, 166, 168–170, 172–176, 178–187, 189–191, 194, 195, 197, 198
5.30r3, 5.31r4	95	as 5.24 without 41 and 181
6.00r2, 6.00r4	93	as 5.30 without 165 and 169
6.03r7	92	0–25, 27–40, 42, 43, 45–49, 67–69, 74, 82, 84, 101, 102, 104–107, 109, 111, 131, 134, 140, 150, 151, 153–156, 166, 168, 170, 172–174, 176, 178–180, 182, 183, 185–187, 189–191, 194, 195, 197, 198

[bin]

Read in order, the id set changes like this:

- 5.00r7 and 5.12 → 5.24 (and the 6.00 betas): drops 75, 132, 133, 171, 193; adds **109** and **198**. So 109 and 198, first noted in Linux 6.01, are already in the early-2010 Windows builds. **[bin]**
- 5.24r4 → 5.30r3: drops 41 and 181. **[bin]**
- 5.31r4 → 6.00r4: drops 165 and 169. **[bin]**
- 6.00r4 → 6.03r7: drops 175 and 184; adds **111** (writes `creationtime.dat`). The Windows 6.03r7 set is identical to the Linux 6.03r7 set in `v6x-clients.md`. **[bin]**

File formats and bundled files

- All builds carry `GR-V1.00`–`GR-V1.05`, `GLEVNW01`, `GWEBL001`, `GRMAP001`, `GRPKG001` and `GRESCR01`, as in 5.00r7. 6.03r7 drops the reader strings for `GR-V1.00`–`GR-V1.02` and `GLEVNW01`. **[bin]**
- The MoleBox bundle is the same small base set in every build: GUI skins under `levels3d/gui/` (`guiblue_*`, `default*`, player-list and PM icons, `graalv4title.png`), translations

`gui_{en,de,fr}.po` and `guiwindows_{en,de,fr}.po`, `tempsitc.ttf`, `fmod.dll`, `ExeReplacer_temp.exe`, two monster ganis and images, one sound, and the list `graalv4_installfiles.txt` (57 lines, identical in all five builds). **[file]**
- The Themida builds read `graal_installfiles.txt` instead and expect the server to supply the files. **[bin][cap]**

Still open

- ~~A server reply~~ and ~~client → server ids beyond 5, 37, 23 and 34~~: answered in `v5v6-lab-play.md` (2026-10-30), from runs against a lab server.
- The expiry check itself (Themida VM code).

`protocol/v5x-clients.md`

Graal 5.x Windows clients: 5.00r7, 5.12, 5.24, 5.30, 5.31

Date: 2026-10-28

Sources: six Windows uploads, read statically. Nothing was run. Only 5.00r7 came unpacked, so most of this page is from that build. Tags: **[bin]** read from the client binaries · **[file]** read from shipped data files · **[inf]** inferred.

What the uploads are

Upload	Contents	Wrapper
5.00r7 "Unpacked, CRC patch"	A complete installed game folder: Delphi launcher `Graal.exe`, engine `GraalEngine.dll`, `Graal3DEngine.dll`, the self-updater `ExeReplacer.exe`, `license/`, `updatepackages/` and caches. Engine and launcher are dated July 2007. It also holds `GraalEngine.dll.bak` (the unpatched engine) and `Graal5 CRC Patch.exe`.	None (unpacked dump)
5.12, 5.24r3, 5.24r4	One setup executable each	Every section has the same name (`.data`, `.udata` or `.CRT$XIA`), the only imports are LoadLibraryA and VirtualAlloc, and an encrypted file bundle sits in the overlay. This looks like a MoleBox-style virtualising wrapper, the same one seen on 4.04r2.
5.30r3 (Oct 2010), 5.31r4 (Jan 2011)	One setup executable each, Authenticode-signed	Themida / WinLicense 2.x: blank and random 8-letter section names, a single import, VC 2008 linker. The Windows 6.00r2+ builds use the same wrapper.

The wrapped builds can't be read without unpacking them, and both wrappers need to run to unpack. So 5.12 to 5.31 are only identified here, not analysed. **[bin]**

The 5.00r7 folder had been used by a player and later updated by the auto-updater: its update packages already name `Graal4.exe` 7.20 and `Graal6.exe` 6.036. **[file]**

Architecture

- This is the same split as 4.20: a **Delphi launcher** (window, DirectDraw/Direct3D 7, DirectInput, Flash panels) and an **MSVC 6 C++ engine DLL**. The DLL holds the game, the network code and the scripting. It

uses the Torque-derived base classes (`GuiControl`, `GuiCanvas`, `MatrixF`, `GBitmap`) and exports about 2,170 decorated C++ names (`TGraalVar`, `TShowImg`, `TScriptExecutionSpace`, `TGraalUniverse`, `TAdventure`, `TParticleEmitter`, `JString`). **[bin]**
- The launcher calls `graal_engineInitialize` and hands the engine its callbacks through the `graal_set*` exports (`graal_setOnServerLogin`, `graal_setOnServerListerConnect`, …). If initialisation fails, the launcher shows "The execution file is corrupted". **[bin]**
- `Graal3DEngine.dll` links against the engine DLL (about 700 imports) and exports only `loadGraal3DEngine`, `runGraal3DEngine`, `endGraal3DEngine`, `resizeGraal3DWindow` and `toggleGraal3DFullScreen`. **[bin]**
- The engine's own source tree is named `graallib`. It contains its own crypto (`crypto/sha1.c`, `rsa.c`, `des.c`, plus Blowfish and MD5), zlib, bzip2, libjpeg, JasPer, giflib 4.0 and fmod. **[bin]**

License file and signatures

- `license/license.graal` is still a CRLF text file: host, port, site name, signup URL, translate URL, then two fallback hosts. The stock values are `<login-host-4x>`, port **14900**, with `loginserver2`/`loginserver3` as fallbacks (the same as 4.20). At least 5 lines are required. Line 2 has a suffix appended. **[bin][file]**
- There are **four files**, as in 4.20 for Windows: `sig.graal` signs the launcher executable, `sig.graallib` signs `GraalEngine.dll`, and `sig.license` signs `license.graal`. Each is RSA-1024 with PKCS#1 v1.5 over SHA-1, e = 17, using the **same public key as 3.x and 4.x**. **[bin]**
- The engine checks the files in this order: all four present, then exe → DLL → license, and it **stops on any failure** ("The license file(s) have not been found." / "The license file is invalid."). The 4.20r8 Windows engine checks the same chain and also stops on failure. **[bin]**

The "CRC patch" (third-party)

- The patched `GraalEngine.dll` differs from the original in **8 bytes**, all in the license loader. **[bin]**
    1. A conditional jump becomes unconditional, so the "files not found" exit is skipped.
    2. A conditional jump becomes unconditional, so the "license is invalid" exit is skipped.
    3. A 6-byte conditional jump becomes NOPs, which removes the "at least 5 lines" check.
- So the patched client accepts any `license.graal`, including a missing or bad signature. In this folder the file had been edited to point at a private LAN host on port 14900, with `.<graalin-site>` fallbacks, which suggests a private or emulated login server. **[file][inf]**
- None of the three signature files in the folder match the files beside them. The signatures decode to a valid SHA-1 DigestInfo, so they belong to other file versions. **[bin][file]**
- `Graal5 CRC Patch.exe` itself is a small Delphi program packed with UPX 0.72, which is too old for current UPX to unpack. It wasn't unpacked; the byte diff above shows what it does. **[bin]**

Login

- **The separate list-server protocol is gone.** The client opens a normal game connection to a server named `Login`, at the host and port from `license.graal`. The server list and account screens are then served by that Login server as scripted GUI. S→C 190 raises the script event `onServerListerConnect`. The 3.x listserver ids (7 `G3D30123newmain`, then 1 account/password) are no longer used. **[bin]**
- **On the Login server the client draws no characters and no default HUD**, only tiles, chests and image NPCs. To get into the game it needs S→C 178 (`PLO_SERVERWARP`, `x,name,host,port`), after which it logs in again to that host under the new name. A server that the client reaches directly through

`license.graal` therefore never shows the player until it sends 178. See `../version-profiles/lab-v5-2026-10-30.md`. **[cap]**

- The first message is id **5**, sent in mode 3 (zlib, no cipher, no type byte): **[bin]**
    1. key GCHAR (`rand & 0x7F`), sent only on the first message
    2. token **`G3D22067`**, assembled from eight one-character globals, so no literal exists
    3. `GCHAR len` + account
    4. `GCHAR len` + password (a password of 0xE0 bytes or more is sent as 0xFF followed by its first 0xDF bytes)
    5. a comma-separated list of four fields: `win`, then the MD5 hex of the Windows `DigitalProductId` registry value, the MD5 hex of the network adapter MAC address (if that isn't available, the node part of `UuidCreateSequential`), and the MD5 hex of the `C:\` volume serial number
- These four fields are **hardware identifiers**, which are new since 3.x. They let the server ban per machine. The 4.20r8 Windows client sends the same layout with token `G3D20126`. **[bin][inf]**
- Next comes id **37**, the language, as in 3.x. Then the socket switches to dynamic mode. **[bin]**
- Id 93 (a long payload) is sent with a GINT3 length through id 50 first. **[bin]**

Cipher

Gen 5, unchanged from 2.22, 2.31, 3.x and 4.20. [bin]

- The LCG is `it = it*0x08088405 + key`, and both directions are seeded with `0x04A80B38`.
- The frame is a `u16be` length, then a type byte in dynamic mode:
    - 2 = raw (under 0x30 bytes)
    - 4 = zlib (under 0x1000 bytes)
    - 6 = bzip2 (larger)
- Type 2 XORs at most 12 dwords; types 4 and 6 XOR at most 4.
- Outgoing data is cut into 1000-byte chunks. A build switch can move the socket onto its own network thread.

Server → client ids (5.00r7)

100 ids are handled: 0–25, 27–43, 45–49, 67, 68, 69, 74, 75, 82, 84, 101, 102, 104–107, 131–134, 140, 150, 151, 153–156, 165, 166, 168–176, 178–187, 189, 190, 191, 193, 194, 195, 197. **[bin]**

Compared with Mac 3.153 (`v3x-clients.md`), 5.00r7 drops 100 and adds the following:

Id	What the handler does
105	Name + GINT5: sets a size or modification time on a cached object
106	Update packages: raises `onUpdatePackageDownloaded` / `onPackagesDownloadComplete`
107	Raises the script level to at least 5, then passes data to a script object
140, 197	Class / weapon script delivery (see GRESCR01 below)
170	GCHAR stored as the connection mode; a mode change resets a player field
182	**Process list request**: the client answers with C→S 44 and the comma list of its running processes
183	**"Is process running?"**: the client checks one process name against the same list and reports back
190	`onServerListerConnect` (the Login server's server-list screen)
191	GSHORT id + data, routed to an object by id
193	GINT5 + name, routed to an NPC (raises the script level to at least 3)

Id	What the handler does
194	Clears every loaded level and the level caches
195	Gani name + gani text, loaded into the named gani (server-sent gani script)

S→C 82 (server text) now also feeds the buddy list (`getbuddylist`, `lister`). Ids 182/183 are an anti-cheat process scan. **[bin][inf]**

Client → server ids (5.00r7)

From the connection's send wrappers: 1, 2, 4, 5, 6, 7, 8, 9, 10, 12–19, 21–24, 26, 28, 29, 32–36, 37, 41, 44, 47, 50, 79, 80, 81, 92, 93, 96, 97, 98, 130, 152, 154–159, 161. Ids sent through a variable aren't captured, so treat this list as a floor. **[bin]**

Ids whose purpose shows from the code that sends them **[bin][inf]**:

Id	Sent when
6	Chat text (the chat bar, including `toall` commands)
9	Arrow shot
12	Item added (sword and shield images)
23	File request
28	Private message
34, 47	File requests used by the update-package downloader (with and without a modification time)
44	Reply to S→C 182/183: the process list
50	Length header in front of a long id 93 payload
80	Request a player profile
81	Set your own profile
152	Request server text
154	Send server text (also used by the options and secure-URL code)
157	Request a gani script
158	Request a weapon or class script (after a GRESCR01 cache miss)
161	Request a class

File formats

Magic	Notes
`GR-V1.00` – `GR-V1.04`	As in 3.x
`GR-V1.05`	New. It is GR-V1.04 with **tile layers**: after the magic comes a GCHAR layer count, then one 64×64 board per layer (13-bit tiles, as in 1.02+). The rest of the body follows as in 1.04. It is accepted in local levels and in the web-level cache. **[bin]**
`GLEVNW01`, `GSERVL01`	Text levels, as in 3.x
`GWEBL001`	Web-level cache, as in 3.x. It now accepts GR-V1.03/1.04/1.05 bodies. **[bin]**
`GRMAP001`	gmap, as in 3.x
`GRESCR01`	New. A **script cache**: the magic, then a body encrypted with **DES-ECB** (8-byte blocks, 8-byte key). On load, the client takes the CRC32 of the plain body and compares it with the value the server sends; on a mismatch the client asks the server for the script again. It caches class and weapon scripts. The key comes from the server with each script (see below). **[bin]**
`GRPKG001`	Update package (`.gupd`), a text file with the keys `NAME`, `VERSION`, `PLATFORM`, `DESCRIPTION`… `DESCRIPTIONEND`, `MODE` (`required`), `ISMAINEXECUTABLE`, `PROTECTOVERWRITE`, `USECHECKSUM`, `SUBPACKAGE`, `FLAG`, `FILE`. Packages nest through `SUBPACKAGE`, with per-platform variants (`_win`, `_mac`, `_linux`). **[file][bin]**

How scripts reach the GRESCR01 cache

- **S→C 140** sends a script. It is a GSHORT header length, then a comma-separated header: the type (`weapon` or `class`), the name, a cache flag and two GINT5 values. The script body follows in **plain text**. **[bin]**
- The two GINT5 values, taken as 32-bit little-endian words, are the **8-byte DES key**. When the cache flag is 1, the client writes `GRESCR01` followed by the DES-encrypted body to disk. It then loads the plain body. Classes go through `TGraalUniverse::addEncryptedClassScript`. **[bin]**
- **S→C 197** tells the client to use its cache. It carries the same key fields plus a CRC32. The client decrypts its cached copy and checks the CRC32 of the plain text. On a match it loads the cached copy. Otherwise it asks for the script again with **C→S 158**. **[bin]**
- So the on-disk encryption only hides scripts from casual reading. The key never stays on the client's disk, but every connection sends it in clear inside the game protocol. **[inf]**

Other files: `game_config.txt` (key=value options, including voice chat, video recording and a fixed UDP port, default 14899), `localversions.txt`, and web caches whose names are percent-encoded with decimal codes (`%058` = `:`, `%047` = `/`). The console log is still `graal4console.log`. **[file][bin]**

Graal3DEngine.dll

- The 3D mode uses **Torque netcode**, as the 3.x clients did: `NetConnection`/`GameConnection`, a connect challenge request (packet type 0x1A), a challenge response, then a connect request. The 3D client logs each step ("Sending Connect challenge Request", "Got Connect Request"). **[bin]**
- The connect request carries a protocol version, and the client accepts **exactly 24**. Any other value is rejected with `CHR_PROTOCOL`; the `CHR_PROTOCOL_LESS`/`_GREATER`/`CHR_GAME`/`CHR_INVALID` reasons are also present. **[bin]**
- User-facing messages include "Only official Graal clients are allowed to connect." and "Please update your Graal, the server is requiring a new Graal version!". The first suggests a class or build check on the server side. **[bin][inf]**
- This is the Torque 3D mode, not the standalone Graal3D protocol in `graal3d/protocol.md` (TCP 14901, `GR3DV002`). **[bin]**

Not done

- ~~Unpacking the 5.12–5.24 bundle and the 5.30/5.31 Themida builds~~ Done 2026-10-29, see `v5v6-windows-unpacked.md`. The 5.12 and 5.24 overlays share a header layout: a size word, the constants 0x29, 0x4823 and 0x18BE, a 16-byte per-build field, then a block common to all three builds.
- Names for the remaining C→S ids
- The 3D server address and the rest of the Torque 3D session

---

`protocol/v6x-clients.md`

Graal 6.x clients: Windows 6.00–6.03, Linux 6.01 and 6.03, Mac OS X 6.11

Date: 2026-10-29

Sources: nine uploads, read statically. Nothing was run. The Linux and Mac builds are unpacked native binaries, so this page is built from them. Tags: **[bin]** read from the client binaries · **[inf]** inferred.

What the uploads are

Upload	What it is	Build stamp
6.00 beta1, beta2 (Windows)	`GraalSetup.exe` is the game itself, behind an obfuscating packer: every section has the same name (`.PATCH` / `.debug`), the only imports are LoadLibraryA and VirtualAlloc, the entry point is junk-jump code, and a ~430 KB payload sits in the overlay. Authenticode-signed (Thawte, Paris).	Jan 2010
6.00r2, 6.00r4, 6.03r7 (Windows)	Themida / WinLicense 2.x (blank and random 8-letter section names, one import, Themida entry stub). VERSIONINFO says `Graal.exe` 6.002, 6.004, 6.037, "Linux Cyberjoueurs".	Jun 2011, Jun 2011, Nov 2012
6.01r1 Linux x64 and x86	ELF, stripped, not packed	18 Sep 2011
6.03r7 Linux x64	ELF, stripped, not packed	11 Nov 2012
6.11r1 Mac OS X	HFS+ disk image, `Graal.app`, Mach-O x86_64, CFBundleVersion 6.111, "©2014 Eurocenter", Developer ID signed. About 1,660 C++ symbols survive (mostly inline and template methods).	25 Mar 2014

Both Windows wrappers need to run to unpack, so the Windows builds are only identified here. 6.01r1 and 6.03r7 Linux are the same releases as Windows 6.01/6.03, so they stand in for them. **[bin][inf]**

Architecture

- One native C++ program per platform. The 4.x/5.x split into a Delphi launcher and a `GraalEngine.dll` is gone. The class names come from the same `graallib` code base as 5.x (`TGraalVar`, `TGraalConnection`, `TGaniObject`, `TServerPlayer`, `GuiGraalCtrl`, `TSecureVar<T>`, `TEncodedFileKey`), but there is no Torque layer left. **[bin]**
- Linux links SDL_mixer, X11 (Xcursor, XF86VidMode), OpenGL and avahi (LAN service discovery). 6.03 moves zlib from 1.2.3 to 1.2.5 and adds PEM/RSA/DSA key handling and `/dev/urandom` use. 6.11 adds TLS, LibTomCrypt (DES, AES, RC4, RSA) and an RTMP/video path. No bzip2 library strings show, but the type-6 frame compressor keeps the bzip2 call shape (block size 5). **[bin]**
- Many short strings (hosts, ports, tokens, URL parts) are stored scrambled and only decoded at run time, so a plain `strings` search does not find them. A per-byte decoder that depends on the string length and the position does the decoding; it is the same in 6.01, 6.03 and 6.11. **[bin]**

Login server

- **No `license.graal` any more.** The 3.x–5.x RSA-signed license file is not read. The login hosts are compiled in, scrambled. **[bin]**
- 6.01r1: `<login-host-4x>`, `<login-host-4x-2>`, `<login-host-4x-3>`, port **14900**, as four scrambled strings. **[bin]**
- 6.03r7: the same three hosts, each stored as `host:14900`, in a list the client walks on failure. A host without a port gets 14900. **[bin]**
- As in 5.x, the client opens a normal game connection to a server named **`Login`**. The server list and account screens then come from that server (S→C 190 raises `onServerListerConnect`). **[bin]**
- 6.11 replaces this with an HTTP "connector", see below. **[bin]**

Login message (6.01 and 6.03)

Id **5**, sent in the mode-3 login frame (zlib, no cipher, no type byte), with the key byte first as before:

1. key byte (random & 0x7F)
2. version token, 8 characters
3. `GCHAR len` + account
4. `GCHAR len` + password (0xFF + the first 0xDF bytes when the length is 0xE0 or more)
5. a comma list: `linux`, then four machine ids

Then id **37** (language), as in 3.x–5.x. **[bin]**

Build	Token	How it is stored
5.00r7 (for comparison)	`G3D22067`	eight one-character globals
6.01r1 Linux	**`G3D0108A`**	`G` plus seven one-character strings, concatenated
6.03r7 Linux	**`G3D0511C`**	one scrambled string `g3d0511c`, decoded and upper-cased
6.11 Mac	set by the server's connector script	—

The Linux machine ids replace 5.x's Windows ones (DigitalProductId, MAC, C: serial): MD5 hex of the `eth0` MAC address, MD5 hex of the inode number of `/etc`, an empty field, and the kernel release + version string. **[bin]**

Guest logins send account `guest` with a password decrypted at run time from 8 compiled-in bytes (DES-style call). **[bin][inf]**

Cipher and framing (classic mode)

- **Gen 5, unchanged.** Both directions are seeded `0x04A80B38`, the LCG is `it = it*0x08088405 + key`, key = rand & 0x7F. Frame type by size: 2 raw (short), 4 zlib, 6 bzip2. XOR at most 12 dwords for type 2 and 4 dwords for types 4/6. Login frame mode 3, then mode 0. **[bin]**
- New since 5.x: a frame of 0xFFFF bytes or more is sent as length `0xFFFF` followed by a 4-byte big-endian length. Buffers flush at 0x1000. **[bin]**
- Packets are still `id+32`, body, `\n`; an outgoing packet that contains a newline is refused with "Illegal characters in the outgoing data". **[bin]**

Server → client ids (6.01, 6.03)

From the game connection's receive dispatcher.

- **6.03r7 (92 ids):** 0–25, 27–40, 42, 43, 45–49, 67, 68, 69, 74, 82, 84, 101, 102, 104–107, 109, 111, 131, 134, 140, 150, 151, 153–156, 166, 168, 170, 172–174, 176, 178–180, 182, 183, 185–187, 189, 190, 191, 194, 195, 197, 198. **[bin]**
- **6.01r1 (91 ids):** the same without 111. **[bin]**
- Against 5.00r7: drops 41, 75, 132, 133, 165, 169, 171, 175, 181, 184, 193; adds 109, 111 and 198. **[bin]**
  - 109: a GINT5 time plus two strings, handed to the file layer (probably a file redirect; the 6.11 native handler list has `setfileredirect(u,s,s)`). **[inf]**
  - 111: writes `creationtime.dat`. **[bin]**
  - 198: a GINT5 value stored on the connection. **[bin]**

Payloads of the new ids **[bin]**:

- **109:** `i5` time (five 7-bit groups), then a c-length string A and a c-length string B. The client passes (B, A, time) to the file layer. Most likely a file redirect (6.11 names a native handler `setfileredirect(u,s,s)`). **[bin][inf]**
- **198:** one `i5`, stored on the connection. Nothing else in 6.03 reads it back.
- **111:** no fields. The receive dispatcher passes the whole packet body to one function that writes it, byte for byte, to `creationtime.dat` in the client's own folder (the same base folder the other cache files use), replacing the file. The client sends nothing back. A matching loader that reads the file into a string exists, but no direct call to it was found, so what reads the value (probably a script builtin) is not known. **[bin] [inf]**

Client → server ids (6.01, 6.03)

All outgoing packets go through one virtual send method `(id, body)`. It writes `id+32`, the body and `\n`. Ids 93 and 163 are special: the client first sends id **50** (`PLI_RAWDATA`) with a 3-byte length (3 × 7-bit groups, capped at 0x37FFFF), then the packet. **[bin]**

- **6.03r7:** 0–20, 22–24, 26–30, 32–38, 41, 44–48, 50, 79, 92, 93, 96–98, 130, 152, 154–162. **[bin]**
- **6.01r1:** the same without 160. **[bin]**
- Not sent as a constant anywhere: 21 (`PUTNPC`), 25, 31 (`PACKETCOUNT`), **39 (`TAMPERCHECK`)** and 40 (`SHOOT`). So 6.x does not send the 10-byte tamper check that 2.22/2.31 send after the language packet. **[bin]**

Id	Name (gs2lib)	6.x use
5	`PLI_BOMBDEL`	also the login message (in the mode-3 frame)
37	`PLI_LANGUAGE`	sent right after login
44, 45	`PLI_PROCESSLIST`, `PLI_PROCESSRUNNING`	replies to the server's process checks
46	`PLI_ENTERLEVEL`	on level entry, as in 2.31
79	`PLI_RC_CHAT`	
92, 93	`PLI_RC_FILEBROWSER_DOWN`, `_UP`	93 carries a file-name string plus one data chunk, preceded by id 50
96, 97, 98	`PLI_RC_FILEBROWSER_MOVE`, `_DELETE`, `_RENAME`	
130	`PLI_REQUESTUPDATEBOARD`	
152, 154	`PLI_REQUESTTEXT`, `PLI_SENDTEXT`	
155, 156	`PLI_RC_LARGEFILESTART`, `_END`	the upload queue: 155, then 93 chunks of at most **32,000 bytes**, then 156. The queue pauses while about 64 KB is already waiting to go out
157, 158, 161	`PLI_UPDATEGANI`, `PLI_UPDATESCRIPT`, `PLI_UPDATECLASS`	script and gani requests
159	`PLI_UPDATEPACKAGEREQUESTFILE`	
160	`PLI_RC_FOLDERDELETE`	6.03 only
162	(no gs2lib name)	**new.** Reply to a server request: `!` followed by a 2-byte value (2 × 7-bit groups, capped at 0x6FFF) the server sent. Probably the ping answer that 6.11 calls `onSendPingAnswer` **[inf]**

The other ids (0–20, 22–24, 26–30, 32–36, 38, 41, 47, 48) have the same meaning as in 2.x–5.x. **[bin][inf]**

The 6.11 named send slots

In 6.11 each outgoing message has a slot in the script-loaded 256-entry table. If a slot holds 0xFE, the client calls a script event instead of sending. The event names give each slot a name **[bin]**:

Slot	Event	Slot	Event	Slot	Event
0	onSendWantWeaponScript	19	onSendTakeExtra	38	onSendLevelWarp
1	onSendWantClassScript	20	onSendRemoveExtra	39	onSendLevelWarpModtime
2	onSendWantGaniScript	21	onSendOpenChest	40	onSendDownloadFile
3	onSendToAllChat	22	onSendShot	41	onSendBoardModify
4	onSendIsPKer	23	onSendBaddyProperties	42	onSendBoardModify2
5	onSendCarryThrown	24	onSendBaddyHurt	43	onSendPreloadLevel
6	onSendAddHorse	25	onSendWeaponImage	44	onSendPlayerProperties
7	onSendDelHorse	26	onSendRCChat	45	onSendNPCProperties
8	onSendAddBomb	27	onSendRequestText	46	onSendFlag
9	onSendDelBomb	28	onSendSetText	47	onSendUnsetFlag
10	onSendServerWarp	29	onSendFileDeletion	48	onSendProjectile
11	onSendTriggerAction	30	onSendFolderDeletion	49	onSendUpdatePackage
12	onSendFireSpying	31	onSendFileRename	50	onSendPrivateMessage
13	onSendExplosion	32	onSendFilesMove	51	onSendUploadStart
14	onSendWeaponHit	33	onSendHaveWindow	52	onSendSaveFile
15	onSendPlayerHurt	34	onSendEnterLevel	53	onSendUploadEnd
16	onSendDeleteNPC	35	onSendWantImage	55	onSendPingAnswer
17	onSendDeleteWeapon	36	onSendImageUpdate	56	onSendWindowList
18	onSendExtra	37	onSendImageUpdateCRC		

Slot 54 and slots above 56 have no event (the login and language packets are sent by the script itself). The slot order is not the wire order, so a 6.11 server must send the table. **[bin]**

6.11 Mac: the connector and a server-defined protocol

This is the big change in 6.x. In 6.11 the client no longer has a fixed protocol.

Connector request

- On login the client sets the server name to `Login` and requests, in order, `https://con.quattroplay.com/con.png`, `https://con.quattroplay.com/con.gs`, `http://con.quattroplay.com/conf.gs`. Each step is retried up to twice, and retries use the host `con2` instead of `con`. After three failures it shows a warning. All URL parts are scrambled strings. **[bin]**
- The query is `?p=` + a comma-joined list `g=<client id>,p=mac,v=<version>,b=<build date and time>`, encrypted with DES-ECB under a compiled-in 8-character key (whole 8-byte blocks only), base64-encoded with the client's own alphabet, then URL-encoded. **[bin]**
- The HTTPS connection checks the server against a certificate bundle compiled into the client (base64, itself DES-encrypted with a second key). The request sends `User-Agent: Graal/<version>`. **[bin]**

Connector reply

The reply is loaded as a script object, `StartScript_Connector`, and its `onCreated` runs. Before that the client sets `scriptip`, `scriptsslcipher`, `scriptsslsubject` and `scriptsslissuer` on it, so the script can check where it came from. If no connector script loads, a built-in `StartScript_Fail` is used. **[bin]**

What the script controls

The connection object gives the script these methods: `connecttogameserver`, `setprotocol`, `sendraw`, `senddata(id, data)`, `flush`, `setsslparameters`, `getsslcipher`/`subject`/`issuer`, `setencryptionparsekey`, `setencryptionout(type, key, iv)`, `getpassword`, `setloginaccount`, `requestbasefiles`, `setoutdatahandlers`, `clearindatahandlers`, `setindatahandlers`, `defaultinhandler(id, data)`, plus `tclient_setencryptionin(type, key, iv)`. **[bin]**

- **Framing.** `setprotocol(template)` switches from classic gen-5 framing to a header the template describes. The header is as long as the template. Each letter marks a field and repeating it sets the width (max 4 bytes, big-endian): `I` packet counter (starts at 1), `L` length, `T` packet id, `E` another field. **[bin]**
- **Encryption.** In that mode the stream can be wrapped in **RC4** or **AES-CBC** (blocks padded with 0xFF), set separately for each direction (`setencryptionout`, `tclient_setencryptionin`). Other names turn it off. TLS is also possible (`setsslparameters`). **[bin]**
- **Client → server ids.** `setoutdatahandlers` loads a 256-entry table: internal message number → wire id. A value of 0xFE sends the message to a script event instead of the wire. **[bin]**
- **Server → client ids.** `setindatahandlers` takes (wire id, handler) pairs. A handler below 0x5F is one of 95 native handlers. 0xFE sends the packet to the script, 0xFF ignores it, and 0xFC/0xFD set two special ids. The native handlers are also exposed to script by name (about 90 `tclient_*` functions, from `tclient_setplayerprops` and `tclient_setlevelboard` to `tclient_setghostsnear` and `tclient_setpackageend`). **[bin]**
- **Login.** The script builds and sends the login itself (`getpassword`, `senddata`). So the token, packet ids, header layout and cipher for 6.11 live in the server's script, not in the client. **[bin][inf]**

The gen-5 code path is still in 6.11 and is the default before a script calls `setprotocol`. **[bin]**

None of this exists in 6.01 or 6.03 (no connector, no RC4/AES, no script-set id tables). **[bin]**

Chat and nickname drawing (6.03)

Read from the Linux 6.03 renderer, for the 6.x Seihai profile. Positions are in tiles from the player's x/y. **[bin]** unless marked.

- **Font.** The default text font is **Tempus Sans ITC, size 24** (the same as 2.0–2.31). On Linux the client maps the name to a font file: `tempus` → `tempsitc`, then `trebuc`, then `courier`, looked up in `levels/fonts/`.
- **Chat text.** White (0xFFFFFF), centred at **x + 1.5**, bottom line at **y − 1.0 + chatoffset** (`chatoffset` defaults to 0; on a horse or when attached the offset is −1.0 instead), and moved up by the player's z height. Chat is queued and drawn after the world ("Game_Draw_Texts"), so it sits on top of everything.
- **Wrapping.** Text of 11 characters or more is word-wrapped to the view width. Breaks fall after one of `.,;+-?!`. Lines are stacked **upward** from the anchor, and each line is centred and clamped to stay on screen.
- **Shadow.** Every text is drawn with a second colour offset by (1, 1). When the text's red or green is above about 0.88 (as with white chat), the second colour has red and green 0 and keeps the text's blue, so white chat gets a blue shadow. Darker text gets a light shadow. **[bin][inf]** (the field order of the draw parameters is inferred.)
- **`showap`.** When the chat text is exactly `showap`, the client shows the player's AP number in its place.
- **Nickname.** Drawn at **x + 1.5, y + 2.5** (below the sprite). Its colour comes from the player's AP through a fixed ramp: 0 or less dark red (64,0,0); 1–26 dull to bright red; 27–49 red fading to white; 50–55 white; 56–70 light green to green-cyan; 71–99 cyan to blue; 100 gold (224,196,0). It is only drawn when the player-list flag allows it, and can be cut to 50 characters by an option.
- **Screen message.** Server text for the whole screen is drawn white, centred, 88 px above the bottom of the view.

Windows builds against Linux

A byte search of the unpacked Windows exes (from `v5v6-windows-unpacked.md`) agrees with the Linux reading **[bin]**:

- 6.00b1, 6.00r2 and 6.03r7 all carry the gen-5 seed and LCG constants and the chat wrap characters `.,;+-?!`, and `showap`.
- `Tempus Sans ITC` and `Game_Draw_Texts` are in 6.00r2 and 6.03r7. 6.00 beta1 has neither; its only capitalised face name is `Arial`, so beta1 probably drew chat in Arial. **[inf]**
- `creationtime.dat` (S→C 111) is in 6.03r7 but not 6.00r2, matching the 6.01 → 6.03 change on Linux.
- None of them has the 6.11 connector (`quattroplay`, `onSend…` events).

File formats

All three builds carry the 5.x set: `GR-V1.03`–`GR-V1.05` levels, `GWEBL001` web-level cache, `GRMAP001` gmaps, `GRPKG001` update packages and `GRESCR01` script cache. No new magic was found. The text-level keywords are not in plain strings in 6.x (probably scrambled like the hosts). **[bin]**

Still open

- ~~Unpacking the Windows builds~~ Done 2026-10-29, see `v5v6-windows-unpacked.md`.
- ~~The client → server id table for 6.01/6.03~~ Done 2026-10-30, above.
- ~~The payload of id 111~~ Done 2026-10-31, above: the body is stored as `creationtime.dat`. Who reads it back is still open.
- A live 6.11 connector script, to recover the real 6.11 token, header and id tables.